

COMSATS UNIVERSITY ISLAMABAD

WAH CAMPUS

Department of Computer Science

DevOps Lab Task

Course Title: DevOps

Course Code: CSC323

Instructor Name: Sir Ibrahim

Class: BSE-8A

Name	Registration No.
Muhammad Sami	SP23-BSE-039

1. Objective

The objective of this lab task is to build a single-node Kubernetes cluster with Minikube on an Ubuntu virtual machine running on AWS EC2, and then to deploy and manage several applications on that cluster. The practical covers launching and preparing the EC2 instance, installing Docker, Minikube and kubectl, deploying Nginx first with kubectl commands and then with YAML manifests, deploying a two-tier WordPress and MySQL application, deploying a to-do list application, and finally verifying that all workloads are running and that Kubernetes automatically replaces a pod that is deleted.

2. Lab Environment

The table below summarises the environment used in this lab. All values are taken from the screenshots in this report. Every command in the report is run over SSH on the EC2 instance as the `ubuntu` user (prompt `ubuntu@ip-172-31-38-26`).

Item	Details
Cloud platform	Amazon Web Services (AWS) EC2, region eu-north-1 (Availability Zone eu-north-1b)
Instance name / ID	sami_kubernetes_lab / i-0e2030e9ad8ded0d6
Instance type	m7i-flex.large (2 vCPU, 8 GiB memory)
Operating system	Ubuntu (selected from the AMI Quick Start list; Minikube reports Ubuntu 26.04)
Storage	30 GiB gp3 root volume
IP addresses	Public IPv4: 16.171.154.62; Private IPv4: 172.31.38.26
Docker	Version 29.1.3 (installed from the docker.io package)
Minikube	v1.39.0 (Docker driver)
kubectl	Client v1.35.9 (Kustomize v5.7.1)
Kubernetes cluster	v1.37.0, single node “minikube” (containerd 2.3.4)

3. Launching the AWS EC2 Instance

The first activity is creating the virtual server that hosts the Kubernetes cluster. The instance is configured in the AWS EC2 “Launch an instance” wizard. In real-world DevOps work, a cloud virtual machine like this is a quick way to obtain a clean, disposable Linux server for building and testing infrastructure.

3.1 Instance Name and Operating System

In this step the student starts the EC2 launch wizard, gives the instance the name `sami_kubernetes_lab` and chooses the operating system image (AMI) for the server. Ubuntu is selected from the Quick Start tab, so the server will run Ubuntu Linux.

Settings shown

- **Name** → `sami_kubernetes_lab`, the label used to identify the instance in the AWS console.

- **Application and OS Images (Amazon Machine Image)** — an AMI contains the operating system and software the instance starts with. The **Ubuntu** tile is highlighted under Quick Start.

Observation

The name field contains `sami_kubernetes_lab` and the Ubuntu option is highlighted, confirming the operating system choice.

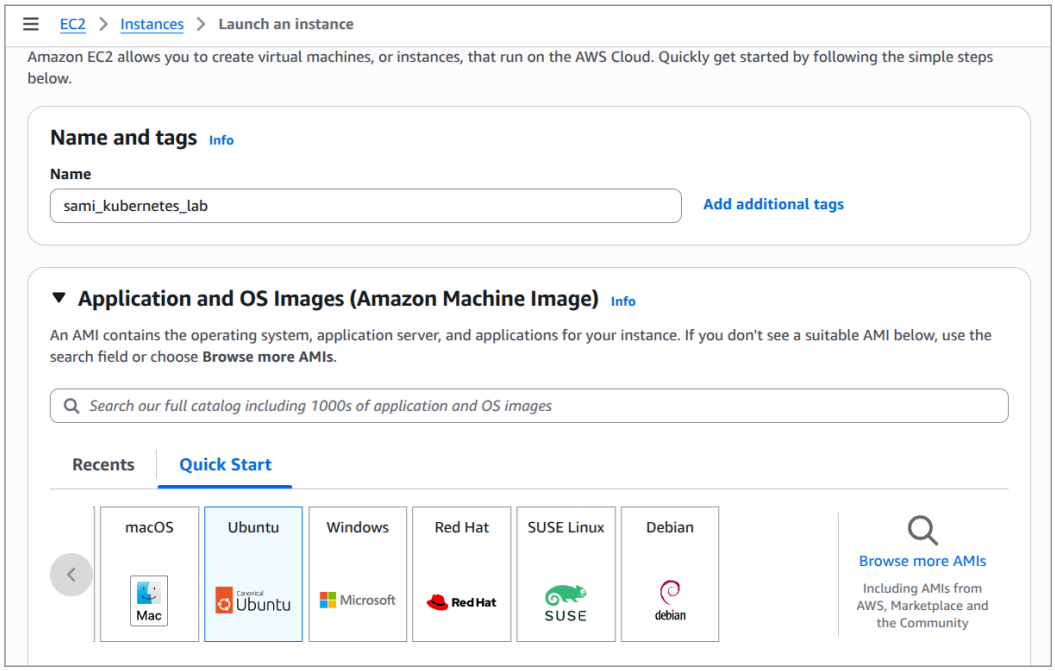


Figure 1: *Launch wizard showing the instance name and the Ubuntu AMI selection*

3.2 Storage Configuration

The student next sets the size and type of the disk attached to the server. Minikube downloads container images and Kubernetes components, so the root disk needs enough free space.

Settings shown

- **1x 30 GiB** — one root volume with a size of 30 GiB.
- **gp3** — the general-purpose SSD volume type.
- **Root volume, 3000 IOPS, Not encrypted** — the properties AWS displays for the volume.

Observation

The Configure storage panel shows a 30 GiB gp3 root volume.



Figure 2: *Configure storage: 30 GiB gp3 root volume*

3.3 Instance Type

The student then chooses the hardware size of the virtual machine. The Kubernetes cluster runs inside this single server, so it needs a reasonable amount of CPU and memory.

Settings shown

- **m7i-flex.large** – the selected instance type, labelled "Free tier eligible".
- **2 vCPU, 8 GiB Memory** – the compute and memory provided by this instance type.

Observation

The Instance type panel shows m7i-flex.large with 2 vCPU and 8 GiB of memory, together with the on-demand pricing information for each operating system.

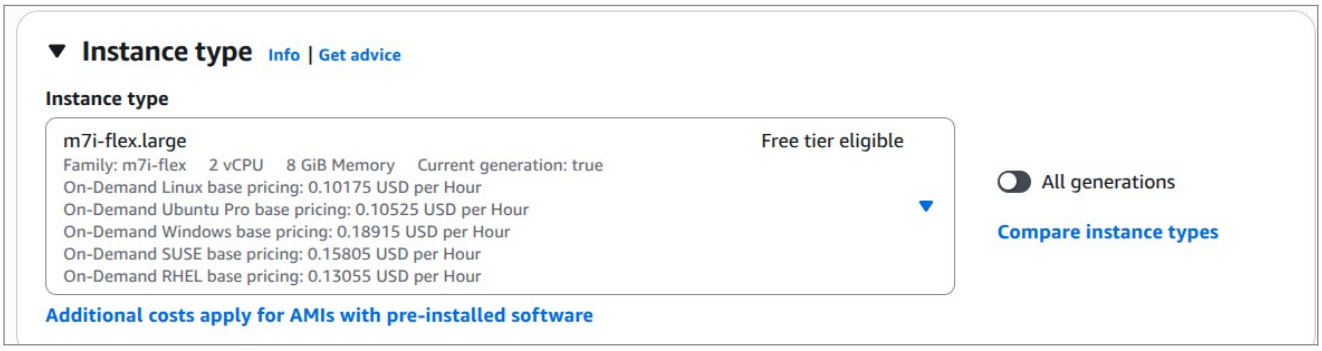


Figure 3: *Instance type selection: m7i-flex.large (2 vCPU, 8 GiB memory)*

3.4 Instance Summary After Launch

After the wizard is completed, the instance appears in the EC2 console. The student opens its summary page to note the details needed in later steps. In particular, the public IPv4 address is used for SSH access and for opening the deployed applications in a web browser.

Details shown

- **Instance ID** – `i-0e2030e9ad8ded0d6` (name `sami_kubernetes_lab`).
- **Instance state** – Running.
- **Public IPv4 address** – `16.171.154.62`, with the public DNS name `ec2-16-171-154-62.eu-north-1.compute.amazonaws.com`.
- **Private IPv4 address** – `172.31.38.26`, which matches the hostname `ip-172-31-38-26` seen in the terminal prompt later in the report.
- **Instance type** – `m7i-flex.large`.

Observation

The instance is in the Running state and has both a public and a private IP address.

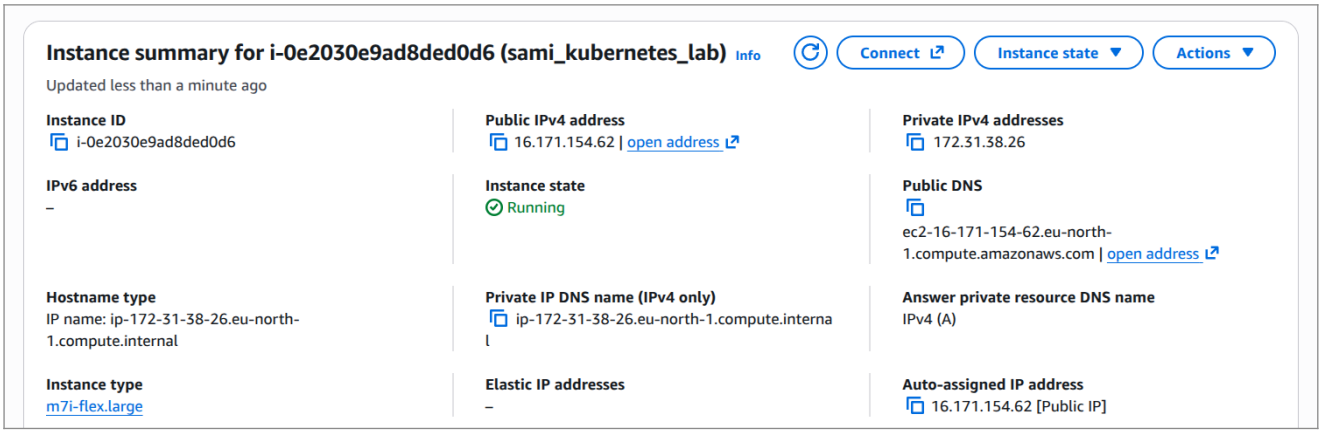


Figure 4: *EC2 instance summary showing the running instance and its IP addresses*

3.5 Security Group Inbound Rules

For the server to be reachable, its security group (the AWS virtual firewall) must allow the required ports. The student reviews the inbound rules attached to the instance. SSH is needed to manage the server, and the extra TCP ports are used later to open the Nginx, WordPress and to-do applications from a browser.

Rules shown

- **SSH, TCP, port 22** – remote terminal access to the server.
- **Custom TCP, ports 8082, 8083, 8081 and 8080** – ports later used for the applications (see the port-forward commands in Sections 7 to 10).

Observation

The Inbound rules list contains five IPv4 rules: port 22 (SSH) and the custom TCP ports 8082, 8083, 8081 and 8080.

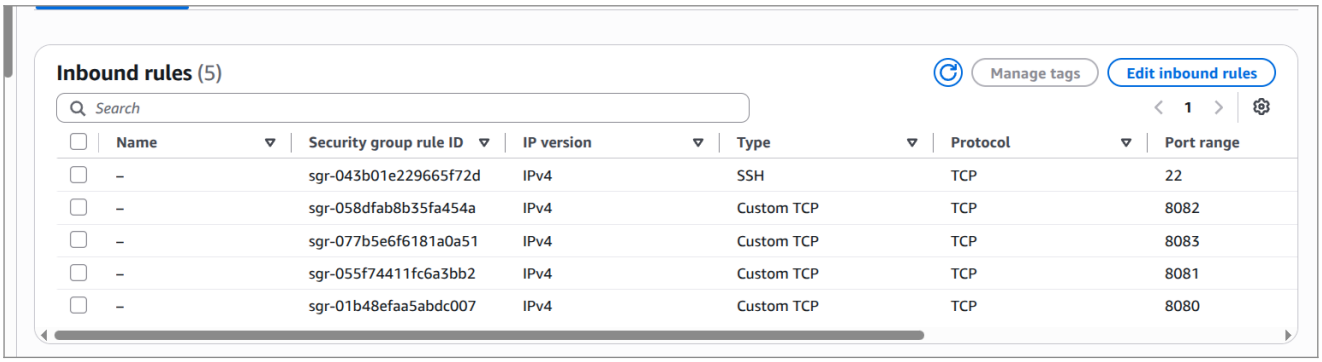


Figure 5: *Security group inbound rules for SSH and application ports 8080–8083*

4. Preparing the Server and Installing Docker

After connecting to the instance over SSH, the student prepares the Ubuntu server. Minikube is started with the Docker driver in this lab, so Docker must be installed first.

4.1 Updating the Package Index

Before installing software, the student refreshes the list of packages available from the Ubuntu repositories. This makes sure the latest package versions and dependency information are used.

Command used

```
sudo apt update
```

Command explanation

- `sudo` → runs the command with administrator (root) privileges.
- `apt update` → downloads the latest package lists from the configured repositories. It does not install or upgrade any package.

Result

The terminal shows `Hit` and `Get` lines for the Ubuntu repositories on `eu-north-1.ec2.archive.ubuntu.com` and `security.ubuntu.com`, which means the package lists are being fetched. The captured part of the output shows the repository downloads; the next screenshot shows that 193 packages can be upgraded.



Figure 6: *Updating the package index with sudo apt update*

4.2 Installing Docker

The student installs Docker from the Ubuntu repository. Docker provides the container runtime that Minikube uses to create the Kubernetes node.

Command used

```
sudo apt install docker.io -y
```

Command explanation

- `apt install docker.io` → installs the `docker.io` package, which is Ubuntu's Docker engine.
- `-y` → automatically answers "yes" to the confirmation prompt so the installation continues without waiting for input.

Result

The output starts with the line "193 packages can be upgraded". It then lists `docker.io` as the package to install together with its dependencies (`bridge-utils`, `containerd`, `dns-root-data`, `dnsmasq-base`, `pigz`, `runc` and `ubuntu-fan`). The summary reports 8 packages to install and a download size of 74.0 MB.

```
193 packages can be upgraded. Run 'apt list --upgradable' to see them.
ubuntu@ip-172-31-38-26:~$ sudo apt install docker.io -y
Installing:
  docker.io

Installing dependencies:
  bridge-utils  containerd  dns-root-data  dnsmasq-base  pigz  runc  ubuntu-fan

Suggested packages:
  ifupdown    cgroupfs-mount  debootstrap    docker-compose-v2  rinse        zfs-fuse
  aufs-tools  | cgroup-lite   docker-buildx  docker-doc         rootlesskit  | zfsutils

Summary:
  Upgrading: 0, Installing: 8, Removing: 0, Not Upgrading: 193
  Download size: 74.0 MB
```

Figure 7: *Installing Docker with sudo apt install docker.io -y*

4.3 Enabling and Verifying Docker

The student makes sure the Docker service is running and checks that the installation works. This step also shows a permission error that is fixed later in the lab.

Commands used

```
sudo systemctl enable --now docker
docker --version
docker ps
sudo docker ps
```

Command explanation

- `sudo systemctl enable --now docker` — `enable` makes the Docker service start automatically at boot and `--now` starts it immediately.
- `docker --version` — prints the installed Docker version.
- `docker ps` — lists the running containers.
- `sudo docker ps` — the same listing, run with administrator privileges.

Result

Docker reports version 29.1.3 (build 29.1.3-0ubuntu4.1), so it is installed. Running `docker ps` as the normal user fails with "permission denied while trying to connect to the docker API at unix:///var/run/docker.sock", meaning the `ubuntu` user is not yet allowed to talk to the Docker daemon. Running the same command with `sudo` works and shows only the column headings, so Docker is running and no containers exist yet. The permission problem is solved in Section 6.1 by adding the user to the `docker` group.

```
No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-38-26:~$ sudo systemctl enable --now docker
ubuntu@ip-172-31-38-26:~$ docker --version
docker ps
Docker version 29.1.3, build 29.1.3-0ubuntu4.1
permission denied while trying to connect to the docker API at unix:///var/run/docker.sock
ubuntu@ip-172-31-38-26:~$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
ubuntu@ip-172-31-38-26:~$
```

Figure 8: *Docker enabled and verified; docker ps fails without sudo and works with sudo*

5. Installing Minikube and kubectl

Minikube creates a local single-node Kubernetes cluster, and kubectl is the command-line tool used to control it. Both are installed on the EC2 server in this section.

5.1 Downloading Minikube

The student downloads the Minikube program for Linux from the official Minikube release storage.

Command used

```
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
```

Command explanation

- `curl` – a tool for transferring data from a URL.
- `-L` – follows redirects if the server moves the file.
- `-O` – saves the file in the current directory using its original name, `minikube-linux-amd64`.

Result

The curl progress table reaches 100% with 136.0 MB received. The transfer took about 4 seconds at an average speed of 29.58 MB/s.

```
ubuntu@ip-172-31-38-26:~$ curl -LO https://storage.googleapis.com/minikube/releases/latest/m
inikube-linux-amd64
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 136.0M 100 136.0M    0     0 29.58M      0   00:04   00:04         28.40M
ubuntu@ip-172-31-38-26:~$
```

Figure 9: *Downloading the Minikube binary with curl*

5.2 Installing Minikube and Checking Its Version

The downloaded file is installed into a system folder so that the `minikube` command can be run from any location. The student then confirms the installation by printing the version.

Commands used

```
sudo install minikube-linux-amd64 /usr/local/bin/minikube
minikube version
```

Command explanation

- `sudo install minikube-linux-amd64 /usr/local/bin/minikube` → copies the downloaded file to `/usr/local/bin` under the name `minikube` and makes it executable.
- `minikube version` → prints the installed Minikube version.

Result

The output reports `minikube version: v1.39.0` together with its commit ID, which shows Minikube is installed and available as a command.



Figure 10: *Installing Minikube to /usr/local/bin and checking the version*

5.3 Checking the Instance Status in the AWS Console

While the tools are being installed, the student returns to the EC2 instance list in the AWS console to confirm that the server is healthy.

Details shown

- **Instance state** → Running.
- **Status check** → 3/3 checks passed.
- **Availability Zone** → `eu-north-1b`.

Observation

The instance `sami_kubernetes_lab` (`m7i-flex.large`) is running and all three status checks have passed.



Figure 11: *EC2 instance list showing the running instance with 3/3 status checks passed*

5.4 Installing kubectl

The student installs kubectl, the tool used to send commands to the Kubernetes cluster, and verifies the client version.

Commands used

```
sudo snap install kubectl --classic
kubectl version --client
```

Command explanation

- `sudo snap install kubectl --classic` → installs kubectl from the Snap store. `--classic` installs it with classic confinement, giving it normal access to the system and to the user's files.
- `kubectl version --client` → prints only the client version and does not contact the cluster.

Result

Snap reports that `kubectl 1.35.9 from Canonical` is installed. The version command shows Client Version v1.35.9 and Kustomize Version v5.7.1.

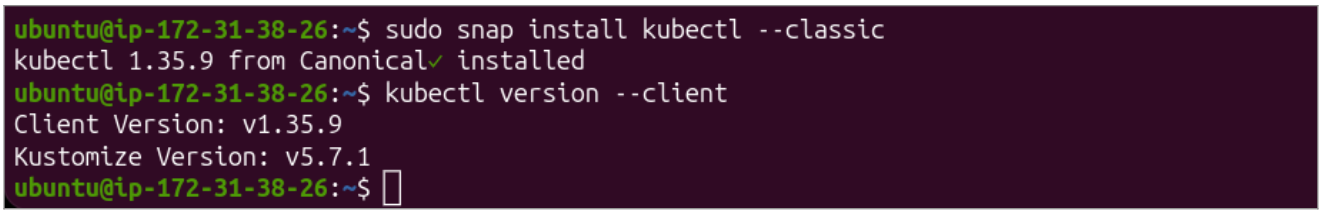


Figure 12: *Installing kubectl with Snap and checking the client version*

5.5 Installing curl

The student installs the `curl` package using apt, so that the transfer tool used earlier is installed and up to date on the server.

Command used

```
sudo apt install curl -y
```

Command explanation

- `apt install curl` → installs the `curl` package (or upgrades it if it is already present).
- `-y` → answers "yes" automatically.

Result

The output lists `curl`, `libcurl3t64-gnutls` and `libcurl4t64` under "Upgrading". The summary shows 3 packages upgraded and 0 newly installed, with a download size of 1120 kB, which means curl was already present and has now been updated.

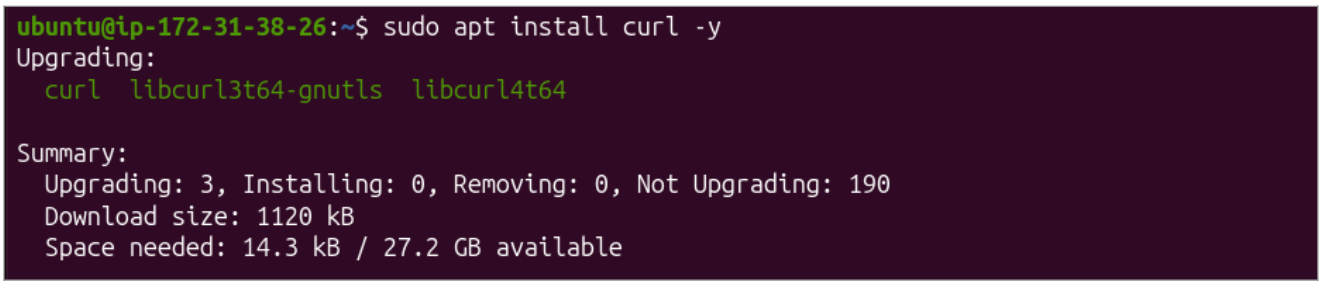


Figure 13: *Installing and updating curl with apt*

6. Starting the Kubernetes Cluster

6.1 Starting Minikube with the Docker Driver

Before starting Minikube, the student fixes the Docker permission error seen in Section 4.3 by adding the `ubuntu` user to the `docker` group. Minikube is then started with Docker as its driver. In real-world environments, tools such as Minikube are used to run a small Kubernetes cluster for development and testing before using a production cluster.

Commands used

```
sudo usermod -aG docker ubuntu
newgrp docker
docker ps
minikube start --driver=docker
```

Command explanation

- `sudo usermod -aG docker ubuntu` → `-G docker` sets the `docker` group and `-a` appends it, so the `ubuntu` user keeps its existing groups.
- `newgrp docker` → starts a shell in which the new group membership is active immediately, without logging out.
- `docker ps` → checks that Docker can now be used without `sudo`.
- `minikube start --driver=docker` → creates the Kubernetes cluster; `--driver=docker` makes Minikube run the Kubernetes node as a Docker container.

Result

`docker ps` now runs without `sudo` and shows only the column headings, so the earlier permission error is resolved. Minikube v1.39.0 starts on Ubuntu 26.04 using the Docker driver, begins the "minikube" control-plane node, pulls the base image and downloads the Kubernetes v1.37.0 preload. The screenshot was captured while the base-image download was at 74.27%; the completed output is shown next.

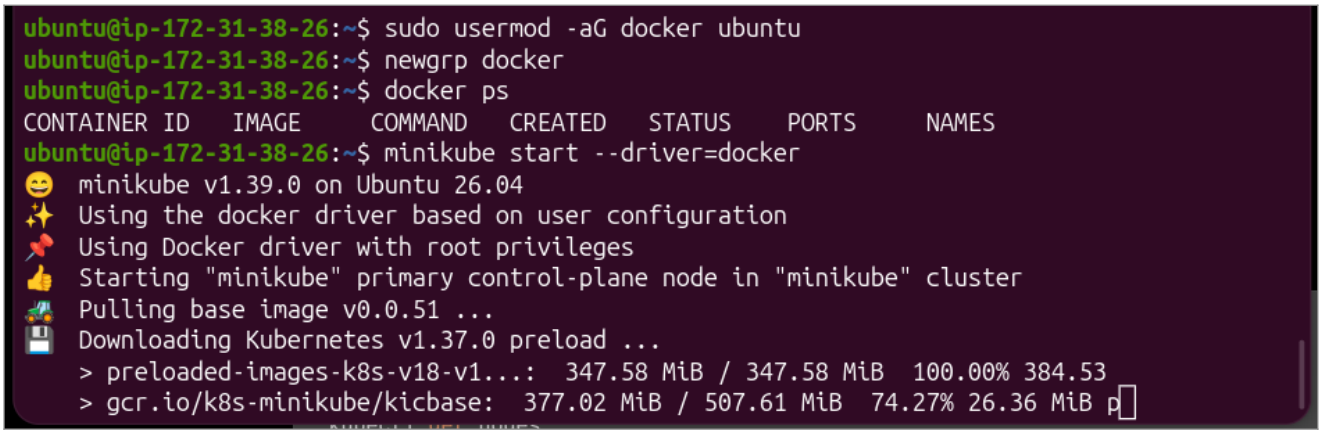


Figure 14: *Adding the user to the docker group and starting Minikube (download in progress)*

6.2 Minikube Start Completed

The same `minikube start --driver=docker` command is shown after it has finished. This output lists each stage of the cluster creation.

Command used

```
minikube start --driver=docker
```

Command explanation

- `minikube start` → creates and starts the local Kubernetes cluster.
- `--driver=docker` → uses Docker to host the cluster node.

Result

Both downloads reach 100%. Minikube creates a Docker container (CPUs=2, Memory=3072MB), prepares Kubernetes v1.37.0 on containerd 2.3.4, configures the CNI (Container Networking Interface) and verifies the Kubernetes components. The addons `default-storageclass` and `storage-provisioner` are enabled. A notice says that kubectl 1.35.9 may have incompatibilities with Kubernetes 1.37.0; it is a version-difference warning, and the output ends with "Done! kubectl is now configured to use 'minikube' cluster and 'default' namespace by default".



Figure 15: *Minikube cluster creation completed successfully*

6.3 Verifying the Cluster

To confirm that the cluster is healthy, the student checks the Minikube status, the Kubernetes node and the cluster information.

Commands used

```
minikube status
kubectl get nodes
kubectl cluster-info
```

Command explanation

- `minikube status` → shows whether the Minikube components are running.
- `kubectl get nodes` → lists the nodes in the Kubernetes cluster.

- `kubectl cluster-info` → shows the addresses of the control plane and core services.

Result

Minikube reports `type: Control Plane` with host, kubelet and apiserver all `Running` and kubeconfig `Configured`. The node `minikube` has status `Ready`, role `control-plane` and version `v1.37.0`. The cluster information shows the control plane at `https://192.168.49.2:8443` and CoreDNS running. The Kubernetes cluster is therefore ready for deployments.

```
ubuntu@ip-172-31-38-26:~$ minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured

ubuntu@ip-172-31-38-26:~$ kubectl get nodes
NAME       STATUS   ROLES           AGE    VERSION
minikube   Ready    control-plane   2m5s   v1.37.0
ubuntu@ip-172-31-38-26:~$ kubectl cluster-info
Kubernetes control plane is running at https://192.168.49.2:8443
CoreDNS is running at https://192.168.49.2:8443/api/v1/namespaces/kube-system/services/kube-
dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
ubuntu@ip-172-31-38-26:~$
```

Figure 16: *Verifying the cluster with minikube status, kubectl get nodes and kubectl cluster-info*

7. Deploying Nginx Using kubectl Commands

The first application is deployed with direct (imperative) kubectl commands. This is the quickest way to create a Deployment and expose it, and it is useful for learning and testing.

7.1 Creating and Exposing the Nginx Deployment

The student creates an Nginx deployment and then exposes it as a service so that it can be reached from outside its pod.

Commands used

```
kubectl create deployment nginx --image=nginx:latest
kubectl expose deployment nginx --type=NodePort --port=80
```

Command explanation

- `kubectl create deployment nginx` → creates a Deployment named `nginx`, which keeps the required pod running.
- `--image=nginx:latest` → the container image to run, here the latest Nginx image.
- `kubectl expose deployment nginx` → creates a Service for the deployment.
- `--type=NodePort` → opens a port on the node so the service can be reached from outside the cluster.
- `--port=80` → the port on which the service listens.

Result

Kubernetes replies `deployment.apps/nginx created` and `service/nginx exposed`, so both objects were created successfully.

```
ubuntu@ip-172-31-38-26:~$ kubectl create deployment nginx --image=nginx:latest
deployment.apps/nginx created
ubuntu@ip-172-31-38-26:~$ kubectl expose deployment nginx --type=NodePort --port=80
service/nginx exposed
ubuntu@ip-172-31-38-26:~$
```

Figure 17: *Creating the nginx deployment and exposing it as a NodePort service*

7.2 Verifying the Deployment, Service and Pod

The student checks that Kubernetes has created the objects and that the pod is running.

Commands used

```
kubectl get deployment nginx
kubectl get service nginx
kubectl get pods
```

Command explanation

- `kubectl get deployment nginx` – shows the deployment status. `READY 1/1` means 1 of the 1 desired replicas is ready.
- `kubectl get service nginx` – shows the service type, internal address and ports.
- `kubectl get pods` – lists the pods and their state.

Result

The deployment `nginx` shows `1/1` ready and available. The service `nginx` is of type `NodePort` with cluster IP `10.111.198.135` and ports `80:30131/TCP`, meaning service port 80 is mapped to node port 30131. The pod `nginx-598cc96cd9-jhf89` is `1/1 Running` with 0 restarts.

```
ubuntu@ip-172-31-38-26:~$ kubectl get deployment nginx
NAME    READY   UP-TO-DATE   AVAILABLE   AGE
nginx   1/1     1            1           76s
ubuntu@ip-172-31-38-26:~$ kubectl get service nginx
NAME    TYPE       CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
nginx   NodePort   10.111.198.135   <none>        80:30131/TCP   72s
ubuntu@ip-172-31-38-26:~$ kubectl get pods
NAME                     READY   STATUS    RESTARTS   AGE
nginx-598cc96cd9-jhf89   1/1     Running   0          104s
ubuntu@ip-172-31-38-26:~$
```

Figure 18: *Nginx deployment, NodePort service and pod verified with kubectl get*

7.3 Accessing Nginx from the Browser

The Minikube cluster runs on its own internal network address (`192.168.49.2`, shown in Section 6.3), so the student uses port forwarding to make the service reachable through the EC2 public IP address. The command keeps running in the terminal while the forward is active.

Command used

```
kubectl port-forward service/nginx 8082:80 --address=0.0.0.0
```

Command explanation

- `kubectl port-forward` – forwards a port on the server to a port of a Kubernetes service.
- `service/nginx` – the service to forward to.
- `8082:80` – server port 8082 is forwarded to port 80 of the service.
- `--address=0.0.0.0` – listens on all network interfaces, not only on localhost, so the application can be opened through the public IP.

Result

The terminal prints `Forwarding from 0.0.0.0:8082 -> 80` and stays active without returning to the prompt, which shows that the port forward is running.

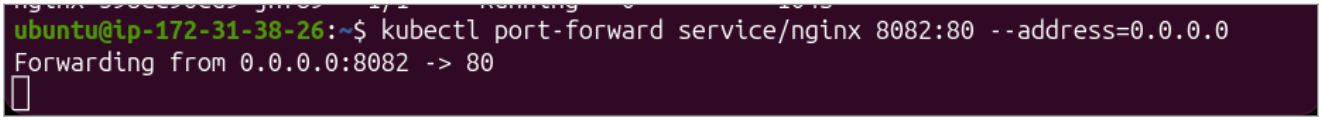


Figure 19: *Port-forwarding the nginx service to port 8082*

With the port forward running, the student opens the EC2 public IP address on port 8082 in a web browser to test the application.

Address opened in the browser

```
16.171.154.62:8082
```

Address explanation

- `16.171.154.62` – public IPv4 address of the EC2 instance.
- `8082` – the forwarded port that reaches the Nginx service.

Observation

The browser displays the default “Welcome to nginx!” page, confirming that Nginx is running in Kubernetes and is reachable from the internet through port 8082.

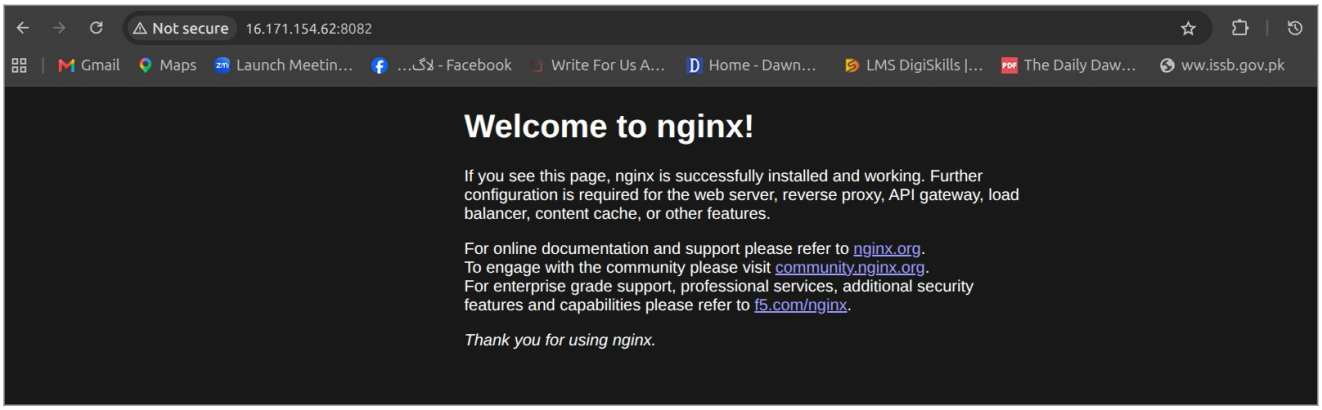


Figure 20: *Nginx welcome page opened at 16.171.154.62:8082*

7.4 Re-checking the Resources After Some Time

Later in the lab, the student repeats the same three `kubectl get` commands to check that the Nginx resources are still healthy.

Commands used

```
kubectl get deployment nginx
kubectl get service nginx
kubectl get pods
```

Command explanation

- The commands are the same as in Section 7.2 and show the deployment, service and pods of `nginx`.

Result

The deployment is still `1/1` ready, the service is still `NodePort` (`80:30131/TCP`) and the pod `nginx-598cc96cd9-jhf89` is still `Running` with 0 restarts. The ages (about 9 minutes) show that the application has been running stably.

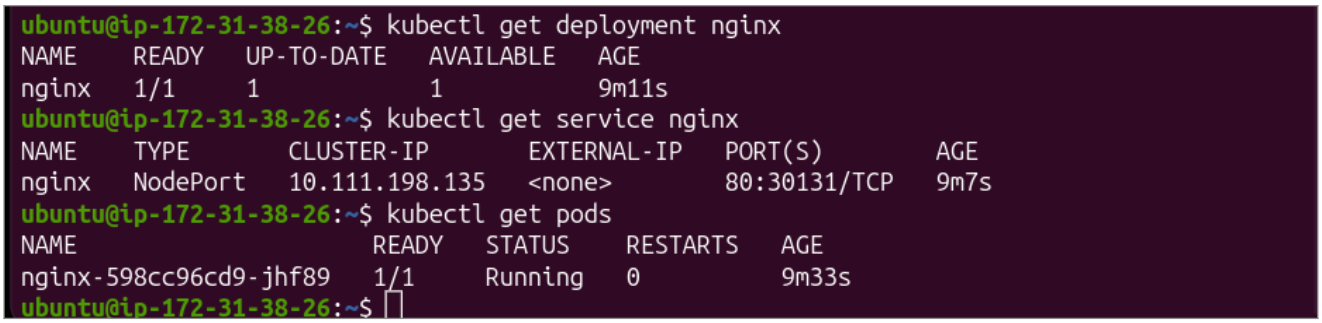


Figure 21: *Nginx deployment, service and pod still running after several minutes*

8. Deploying Nginx Using YAML Manifests

The next deployment uses YAML manifest files. This declarative approach describes the desired state of the application in files, which can be stored, reviewed and reused. In real-world DevOps workflows, manifests like these are normally kept in version control.

8.1 Creating the Deployment Manifest

The student writes the Deployment manifest `nginx-dep.yml` in the nano text editor (the editing commands are shown in Section 8.3). The file asks Kubernetes to run four copies of an Nginx container.

Key fields in the file

- `kind: Deployment` and `name: nginx-dep` – the type and name of the resource.
- `replicas: 4` – four identical pods must be kept running.
- `selector.matchLabels` – `app: nginx-kube`; the deployment manages pods that carry this label.
- `template.metadata.labels` – `app: nginx-kube`; the label given to every pod created from the template.
- `containers` – one container named `nginx` using the image `nginx:1.14.2` and `containerPort: 80`.

Observation

The editor shows the complete manifest, ending with `containerPort: 80`. The title bar shows `nginx-dep.yml`, and the asterisk means the file has not been saved yet.

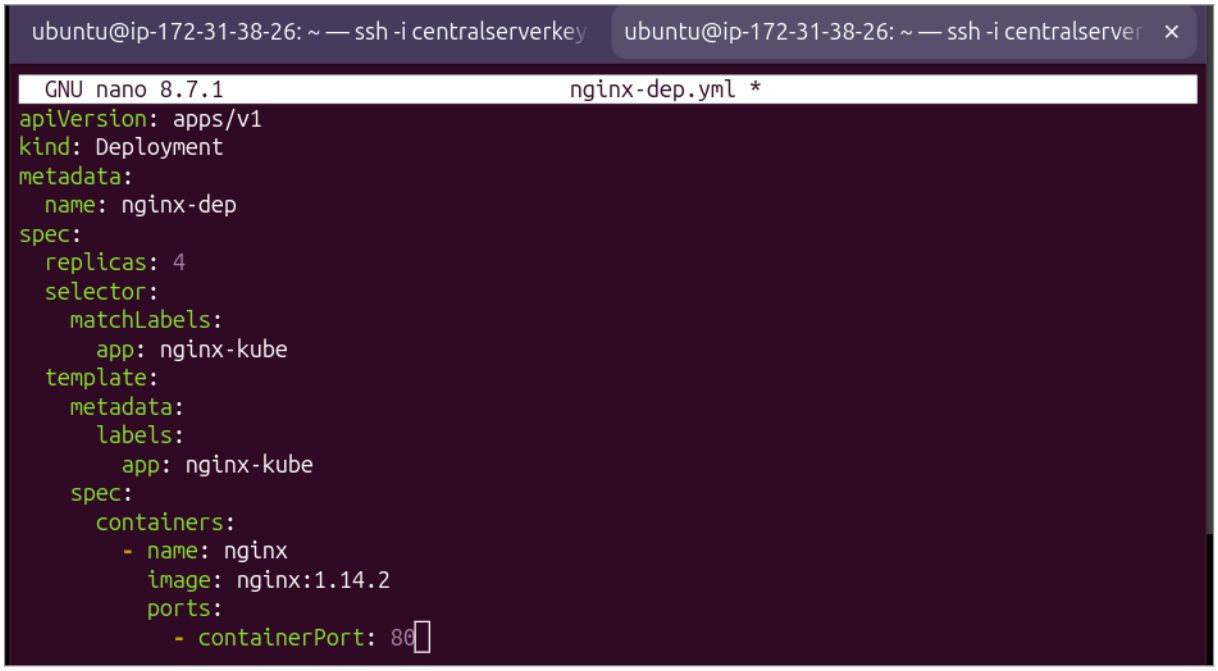


Figure 22: *Deployment manifest nginx-dep.yml in the nano editor*

8.2 Creating the Service Manifest

The student then writes the Service manifest `nginx-svc.yml`, which exposes the four Nginx pods through a single service.

Key fields in the file

- `kind: Service` and `name: nginx-svc` → the type and name of the resource.
- `type: LoadBalancer` → requests an external load-balanced service.
- `port: 80` and `targetPort: 80` → the service listens on port 80 and sends traffic to port 80 of the pods.
- `selector` → `app: nginx-kube`; the service sends traffic to pods with this label, which are the pods created by `nginx-dep`.

Observation

The editor shows the complete `nginx-svc.yml` manifest. The label in the selector matches the label used in the deployment manifest.

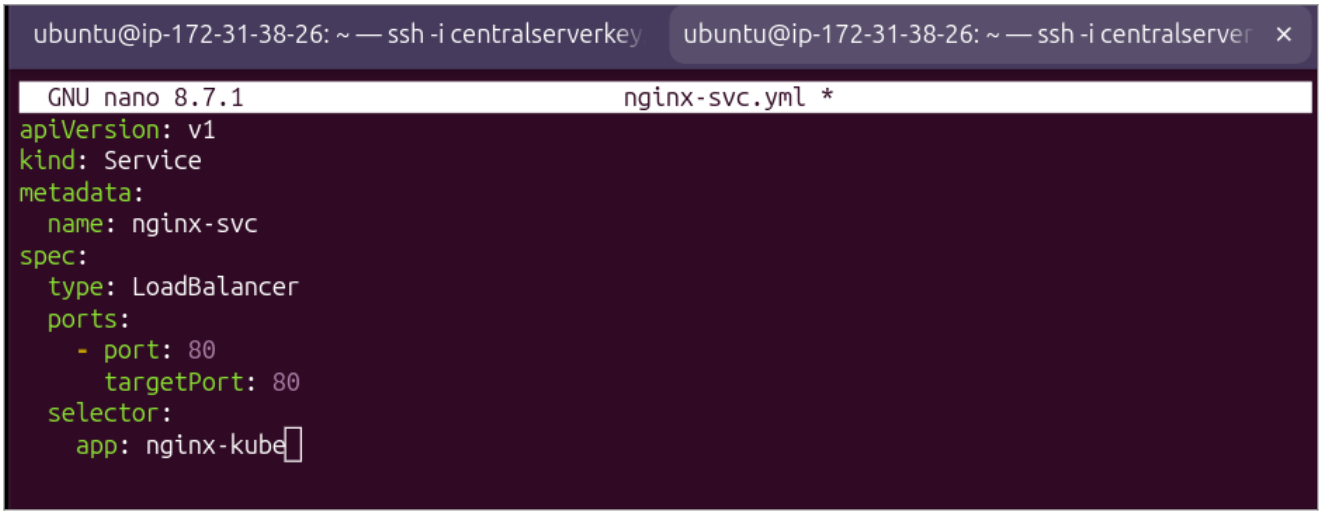


Figure 23: *Service manifest nginx-svc.yml in the nano editor*

8.3 Applying the Manifests

After saving both files, the student applies them to the cluster so Kubernetes creates the resources they describe.

Commands used

```
nano nginx-dep.yml
nano nginx-svc.yml
kubectl apply -f nginx-dep.yml
kubectl apply -f nginx-svc.yml
```

Command explanation

- `nano nginx-dep.yml` and `nano nginx-svc.yml` → open the two manifest files in the nano editor.
- `kubectl apply -f nginx-dep.yml` → creates or updates the resources defined in the file; `-f` gives the file name.
- `kubectl apply -f nginx-svc.yml` → does the same for the service file.

Result

Kubernetes responds with `deployment.apps/nginx-dep created` and `service/nginx-svc created`, so both resources were created from the manifests.

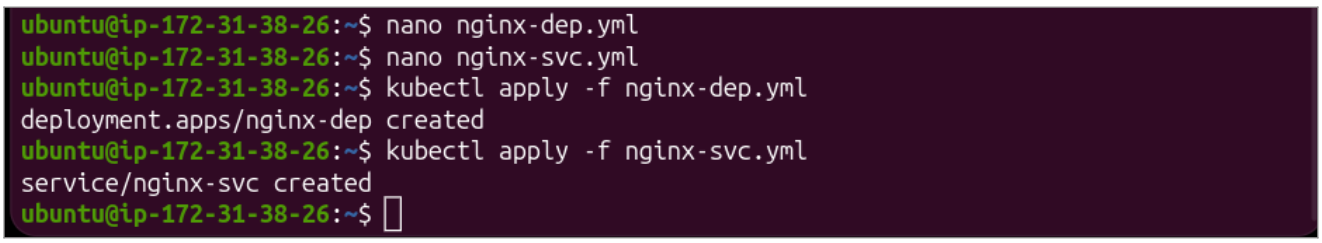


Figure 24: *Applying nginx-dep.yml and nginx-svc.yml with kubectl apply*

8.4 Verifying the YAML Deployment

The student checks the new deployment, service and pods. The `-l` option filters pods by label, so only the pods created by this deployment are listed.

Commands used

```
kubectl get deployment nginx-dep
kubectl get service nginx-svc
kubectl get pods -l app=nginx-kube
```

Command explanation

- `kubectl get deployment nginx-dep` → shows the deployment status.
- `kubectl get service nginx-svc` → shows the service details.
- `kubectl get pods -l app=nginx-kube` → `-l` selects pods that have the label `app=nginx-kube`.

Result

The deployment `nginx-dep` shows `4/4` ready. The service `nginx-svc` is of type `LoadBalancer` with cluster IP `10.109.177.192`, ports `80:30405/TCP` and an external IP of `<pending>`, which means no external address has been assigned, so the application is reached through port forwarding in the next step. Four pods named `nginx-dep-5fd6b8bd65-...` are `1/1 Running` with 0 restarts, matching the four replicas in the manifest.

```
ubuntu@ip-172-31-38-26:~$ kubectl get deployment nginx-dep
NAME        READY   UP-TO-DATE   AVAILABLE   AGE
nginx-dep   4/4     4            4           52s
ubuntu@ip-172-31-38-26:~$ kubectl get service nginx-svc
NAME        TYPE           CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
nginx-svc   LoadBalancer   10.109.177.192   <pending>     80:30405/TCP   52s
ubuntu@ip-172-31-38-26:~$ kubectl get pods -l app=nginx-kube
NAME                         READY   STATUS    RESTARTS   AGE
nginx-dep-5fd6b8bd65-4cw4d   1/1     Running   0          76s
nginx-dep-5fd6b8bd65-bmb9g   1/1     Running   0          76s
nginx-dep-5fd6b8bd65-m98rl   1/1     Running   0          76s
nginx-dep-5fd6b8bd65-z5q95   1/1     Running   0          76s
ubuntu@ip-172-31-38-26:~$
```

Figure 25: *Verifying nginx-dep, nginx-svc and the four running pods*

8.5 Accessing the YAML-Deployed Nginx in the Browser

To open the new service from a browser, the student forwards server port 8080 to port 80 of the `nginx-svc` service.

Command used

```
kubectl port-forward service/nginx-svc 8080:80 --address=0.0.0.0
```

Command explanation

- `service/nginx-svc` → the service to forward to.
- `8080:80` → server port 8080 is forwarded to port 80 of the service.
- `--address=0.0.0.0` → listens on all network interfaces so the page can be opened through the public IP.

Result

The terminal prints `Forwarding from 0.0.0.0:8080 -> 80` and keeps running.

```
ubuntu@ip-172-31-38-26:~$ kubectl port-forward service/nginx-svc 8080:80 --address=0.0.0.0
Forwarding from 0.0.0.0:8080 -> 80
```

Figure 26: *Port-forwarding the nginx-svc service to port 8080*

The student opens the EC2 public IP address on port 8080 in the browser.

Address opened in the browser

```
16.171.154.62:8080
```

Address explanation

- `16.171.154.62:8080` — public IP of the EC2 instance and the forwarded port for `nginx-svc`.

Observation

The default "Welcome to nginx!" page is displayed, confirming that the application deployed from the YAML manifests is reachable.

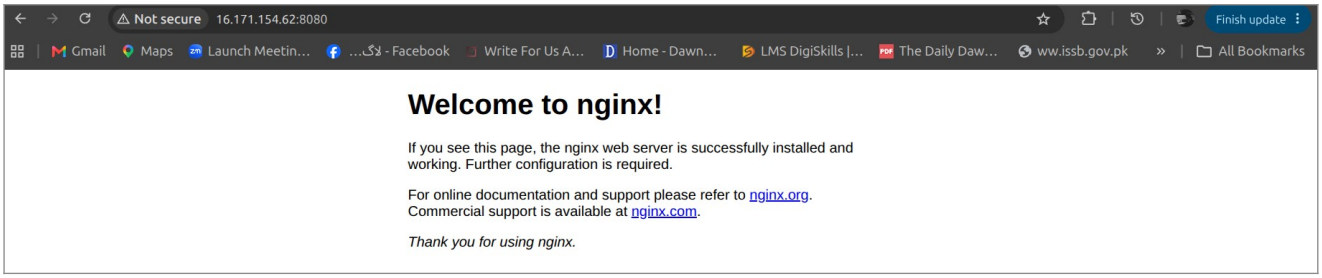


Figure 27: *Nginx welcome page opened at 16.171.154.62:8080*

8.6 Observing Kubernetes Self-Healing

Because the port forward occupies the first terminal, the student opens a second SSH session to the same server. The login banner of the new session is shown, followed by a command that lists the pods of the YAML deployment. The `-l` option selects pods by label.

Command used

```
kubectl get pods -l app=nginx-kube
```

Command explanation

- `kubectl get pods -l app=nginx-kube` — lists only the pods that carry the label `app=nginx-kube`.

Result

The login banner shows the server's IPv4 address `172.31.38.26`. The command lists four pods (`4cw4d`, `bmb9g`, `m98rl` and `z5q95` at the end of their names), all `1/1 Running` with 0 restarts and an age of about 5 minutes.

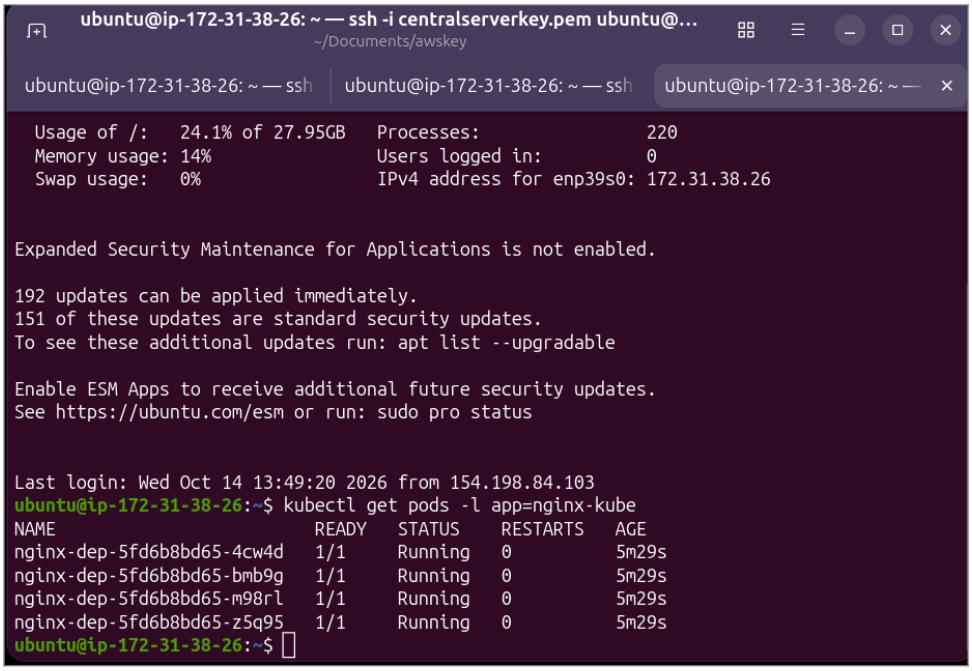


Figure 28: *Second SSH session listing the four nginx-dep pods*

Next, the student tests the self-healing behaviour of Kubernetes. One of the four pods is deleted manually, and the pods and deployment are listed again to see whether Kubernetes restores the desired number of replicas.

Commands used

```
kubectl delete pod nginx-dep-5fd6b8bd65-4cw4d
kubectl get pods -l app=nginx-kube
kubectl get deployment nginx-dep
```

Command explanation

- `kubectl delete pod nginx-dep-5fd6b8bd65-4cw4d` → deletes the pod with this name.
- `kubectl get pods -l app=nginx-kube` → lists the pods of the deployment again.
- `kubectl get deployment nginx-dep` → shows how many replicas are ready.

Result

Kubernetes confirms `pod "nginx-dep-5fd6b8bd65-4cw4d" deleted from default namespace`. The new pod list still contains four pods: the deleted pod is gone and a new pod, `nginx-dep-5fd6b8bd65-w6jw8`, appears with an age of only 11 seconds. The deployment still shows `4/4` ready. This demonstrates that the Deployment automatically creates a replacement pod to maintain the four replicas required by the manifest.



Figure 29: *Deleting a pod and Kubernetes automatically creating a replacement*

9. Deploying WordPress with MySQL

This section deploys a two-tier application: a WordPress web front end and a MySQL database. Each tier gets a Deployment and a Service. In real-world setups, splitting the application and the database into separate workloads allows each part to be managed and scaled independently.

9.1 Creating the WordPress Manifest

The student creates the file `wordpress.yml` in nano. It contains two Kubernetes resources separated by a line of three dashes (`---`): a Service and a Deployment for the WordPress front end. The file is shown in two screenshots, first its upper part and then its lower part.

Key fields in the file

- **Service `wordpress-frontend`** – type `LoadBalancer`, port 80 forwarded to `targetPort` 80, selecting pods with `app: wordpress-frontend`.
- **Deployment `wordpress-frontend`** – `replicas: 1`, so a single WordPress pod is run.
- **Container `wordpress`** – uses the image `wordpress:6.2.1-apache` and `containerPort: 80`.
- **Environment variables (`env`)** – `WORDPRESS_DB_HOST` is set to `wordpress-mysql:3306`, the name and port of the MySQL service, so WordPress can find its database. `WORDPRESS_DB_USER`, `WORDPRESS_DB_PASSWORD` and `WORDPRESS_DB_NAME` provide the database login details and database name.

Observation

The first screenshot shows the Service and the beginning of the Deployment; the second shows the pod template with the container, the database environment variables and the container port.

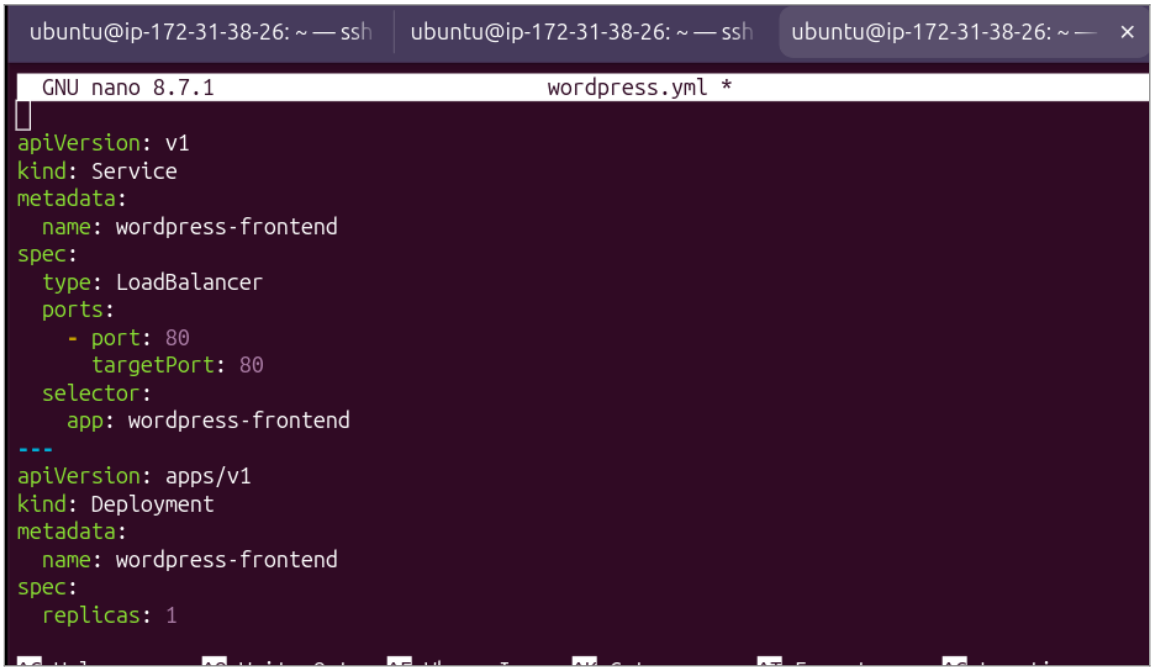


Figure 30: *wordpress.yml (upper part): wordpress-frontend Service and the start of the Deployment*

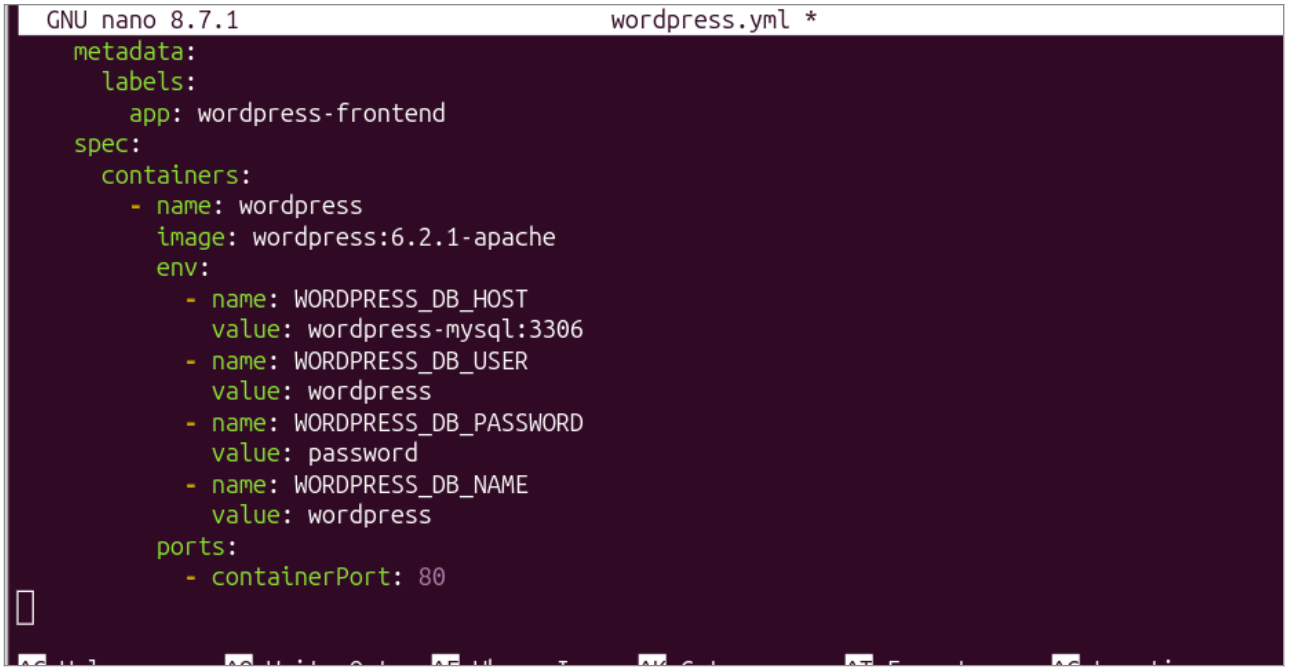


Figure 31: *wordpress.yml (lower part): container image, database environment variables and port*

9.2 Creating the MySQL Manifest

Next, the student creates `mysql.yml` for the database tier. The screenshot shows the upper part of the file; the remaining lines of the Deployment are below the visible area of the editor.

Key fields in the file

- **Service `wordpress-mysql`** → port 3306 forwarded to `targetPort` 3306, selecting pods with `app: wordpress-mysql`. No `type` is written, so Kubernetes uses the default internal type (ClusterIP), which is confirmed in the service list in Section 9.4.
- **Deployment `wordpress-mysql`** → `replicas: 1`, so a single database pod is run.

Observation

The editor shows `mysql.yml` with the service definition and the start of the deployment definition.

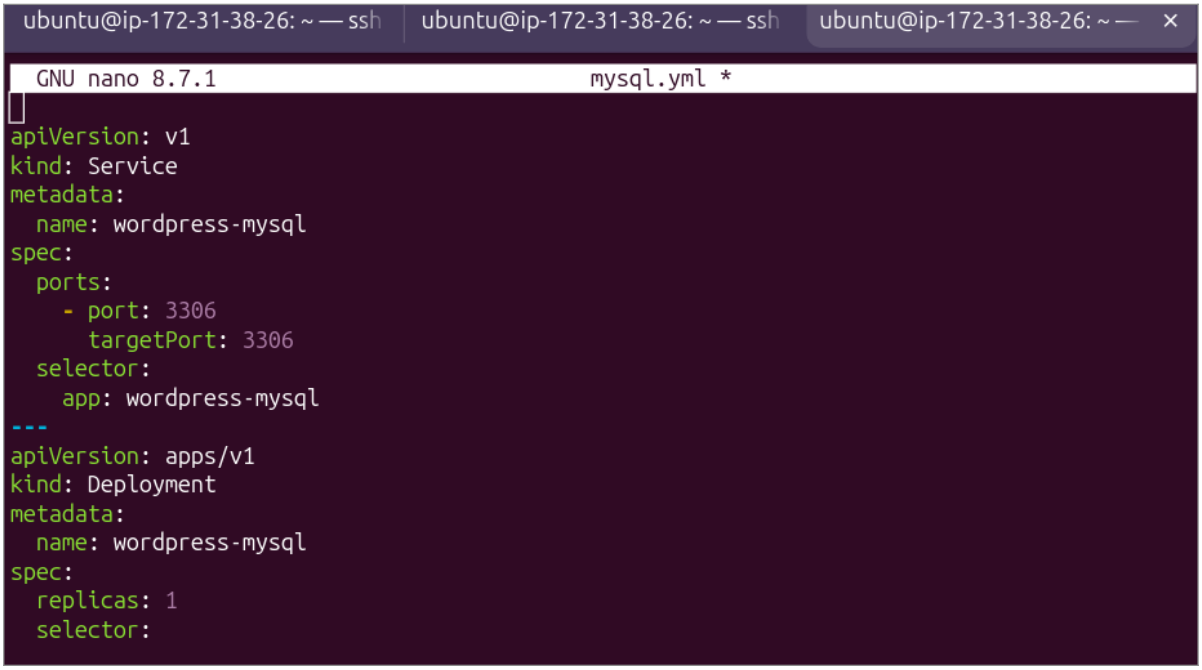


Figure 32: *mysql.yml in the nano editor: wordpress-mysql Service and Deployment*

9.3 Applying the WordPress and MySQL Manifests

The student applies the database manifest first and then the WordPress manifest, so the database service exists before the WordPress front end starts.

Commands used

```
nano wordpress.yml
nano mysql.yml
kubectl apply -f mysql.yml
kubectl apply -f wordpress.yml
```

Command explanation

- `nano wordpress.yml` and `nano mysql.yml` → open the two manifest files in the nano editor.
- `kubectl apply -f mysql.yml` → creates the resources defined in `mysql.yml`.
- `kubectl apply -f wordpress.yml` → creates the resources defined in `wordpress.yml`.

Result

Kubernetes creates `service/wordpress-mysql` and `deployment.apps/wordpress-mysql`, and then `service/wordpress-frontend` and `deployment.apps/wordpress-frontend`.

```
ubuntu@ip-172-31-38-26:~$ nano wordpress.yml
ubuntu@ip-172-31-38-26:~$ nano mysql.yml
ubuntu@ip-172-31-38-26:~$ kubectl apply -f mysql.yml
service/wordpress-mysql created
deployment.apps/wordpress-mysql created
ubuntu@ip-172-31-38-26:~$ kubectl apply -f wordpress.yml
service/wordpress-frontend created
deployment.apps/wordpress-frontend created
ubuntu@ip-172-31-38-26:~$
```

Figure 33: *Applying mysql.yml and wordpress.yml*

9.4 Verifying the WordPress and MySQL Resources

The student lists the deployments in the cluster to confirm that the new WordPress and MySQL deployments are ready.

Command used

```
kubectl get deployments
```

Command explanation

- `kubectl get deployments` → lists all deployments in the current namespace.

Result

Four deployments are listed: `nginx` (1/1), `nginx-dep` (4/4), `wordpress-frontend` (1/1) and `wordpress-mysql` (1/1). The two new deployments are ready and are only seconds old.

```
ubuntu@ip-172-31-38-26:~$ kubectl get deployments
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
nginx               1/1     1            1           25m
nginx-dep           4/4     4            4           13m
wordpress-frontend  1/1     1            1           34s
wordpress-mysql     1/1     1            1           46s
```

Figure 34: *Deployments listed, including wordpress-frontend and wordpress-mysql*

The student then lists the pods to confirm that the pods behind all deployments are running.

Command used

```
kubectl get pods
```

Command explanation

- `kubectl get pods` → lists all pods in the current namespace together with their status.

Result

All pods are `1/1 Running` with 0 restarts. They include the original nginx pod, the four `nginx-dep` pods (including the replacement pod `w6jw8` created earlier), `wordpress-frontend-6c76bd44-bwcx8` and `wordpress-mysql-5bc89547d7-tm76v`.

```
ubuntu@ip-172-31-38-26:~$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
nginx-598cc96cd9-jhf89              1/1     Running   0          26m
nginx-dep-5fd6b8bd65-bmb9g          1/1     Running   0          13m
nginx-dep-5fd6b8bd65-m98rl          1/1     Running   0          13m
nginx-dep-5fd6b8bd65-w6jw8          1/1     Running   0          6m5s
nginx-dep-5fd6b8bd65-z5q95          1/1     Running   0          13m
wordpress-frontend-6c76bd44-bwcx8   1/1     Running   0          48s
wordpress-mysql-5bc89547d7-tm76v    1/1     Running   0          60s
ubuntu@ip-172-31-38-26:~$
```

Figure 35: *All pods running, including the WordPress and MySQL pods*

Finally, the student lists the services to check how the WordPress and MySQL tiers are exposed.

Command used

```
kubectl get services
```

Command explanation

- `kubectl get services` → lists all services with their type, internal IP address and ports.

Result

The list shows `wordpress-frontend` as a `LoadBalancer` service on `80:31766/TCP` (external IP `<pending>`) and `wordpress-mysql` as a `ClusterIP` service on `3306/TCP`, so the database is reachable only inside the cluster. The services `kubernetes`, `nginx` and `nginx-svc` from the earlier steps are also listed.

```
ubuntu@ip-172-31-38-26:~$ kubectl get services
NAME                TYPE           CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
kubernetes          ClusterIP      10.96.0.1        <none>        443/TCP        31m
nginx               NodePort       10.111.198.135   <none>        80:30131/TCP   27m
nginx-svc           LoadBalancer   10.109.177.192   <pending>     80:30405/TCP   14m
wordpress-frontend  LoadBalancer   10.111.210.19    <pending>     80:31766/TCP   2m2s
wordpress-mysql     ClusterIP      10.111.191.104   <none>        3306/TCP       2m14s
ubuntu@ip-172-31-38-26:~$
```

Figure 36: *Services listed: wordpress-frontend (LoadBalancer) and wordpress-mysql (ClusterIP)*

9.5 Opening WordPress in the Browser

The student lists the deployments once more and then forwards server port 8081 to the WordPress service so that the site can be opened from a browser.

Commands used

```
kubectl get deployments
kubectl port-forward service/wordpress-frontend 8081:80 --address=0.0.0.0
```

Command explanation

- `kubectl get deployments` → confirms the deployments are still ready.
- `kubectl port-forward service/wordpress-frontend 8081:80` → forwards server port 8081 to port 80 of the `wordpress-frontend` service.
- `--address=0.0.0.0` → listens on all network interfaces so the site is reachable via the public IP.

Result

All four deployments are ready (`1/1`, `4/4`, `1/1`, `1/1`). The terminal prints `Forwarding from 0.0.0.0:8081 -> 80` and keeps running.

```
ubuntu@ip-172-31-38-26:~$ kubectl get deployments
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
nginx               1/1     1            1           30m
nginx-dep           4/4     4            4           18m
wordpress-frontend  1/1     1            1           5m11s
wordpress-mysql     1/1     1            1           5m23s
ubuntu@ip-172-31-38-26:~$ kubectl port-forward service/wordpress-frontend 8081:80 --address=
0.0.0.0
Forwarding from 0.0.0.0:8081 -> 80
```

Figure 37: *Port-forwarding the wordpress-frontend service to port 8081*

The student opens the WordPress installer through the forwarded port. The first page asks which language to use for the installation.

Address opened in the browser

> `16.171.154.62:8081/wp-admin/install.php`

Address explanation

- `16.171.154.62:8081` → public IP and forwarded port of the WordPress service.
- `/wp-admin/install.php` → the WordPress installation page.

Observation

The WordPress installer loads and shows the language list with English (United States) selected. This shows that the WordPress pod is running and that it can serve pages.

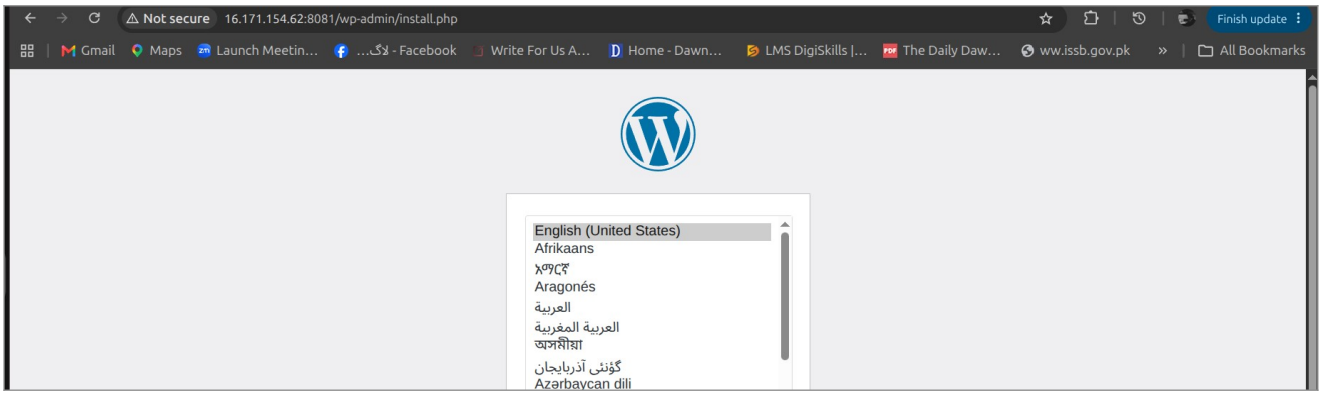


Figure 38: *WordPress installer language selection page*

9.6 Completing the WordPress Installation

After the language is chosen, WordPress shows its “Welcome” page. The site information is entered on this page: the site title, an administrator username, a password and an email address.

Fields on the page

- **Site Title**, **Username**, **Password** and **Your Email** → the details required to create the first administrator account.

- WordPress suggests a generated password and rates it as “Strong”.

Observation

The “Information needed” form is displayed with its fields still empty and a generated password suggestion.

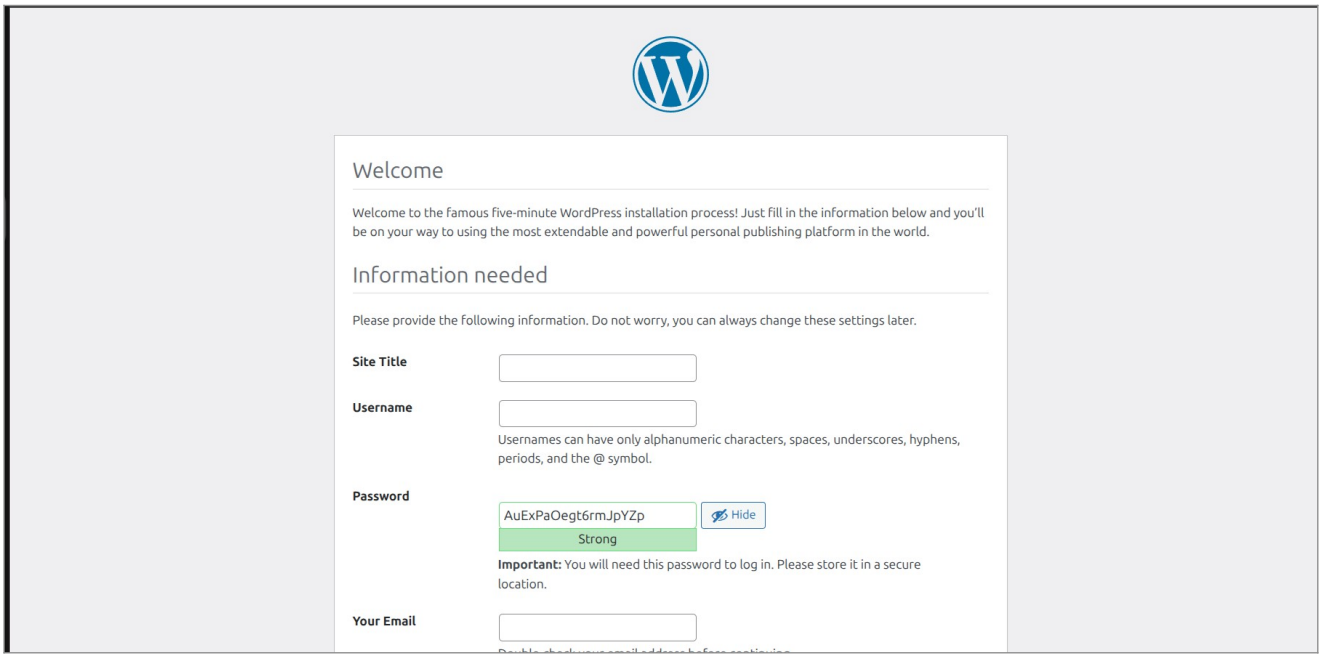


Figure 39: *WordPress “Welcome” page with the installation form*

The student fills in the form. The site title is “Kubernetes Lab Website” and the username is `wpadmin_lab`. A different password is entered, which WordPress rates as weak, so the option to confirm its use is ticked. The search-engine visibility option is left unticked.

Fields on the page

- **Password strength** — shown as “Weak”, so WordPress requires “Confirm use of weak password” to be ticked.
- **Discourage search engines from indexing this site** — left unticked.

Observation

The form contains the site title, the username, a masked password with a “Weak” rating, the confirmation checkbox ticked and an email address.

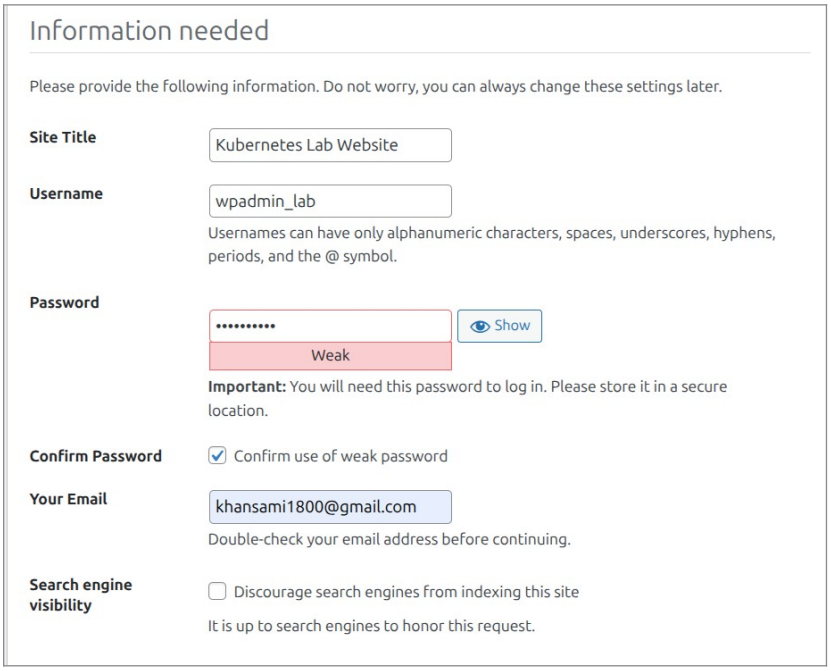


Figure 40: *Installation form completed with site title, username and password*

The student submits the form to finish the installation.

Observation

The address changes to `install.php?step=2` and WordPress shows "Success! WordPress has been installed." together with the username `wpadmin_lab` and a Log In button. The WordPress and MySQL pods are therefore working together: WordPress could write its installation data to the MySQL database.

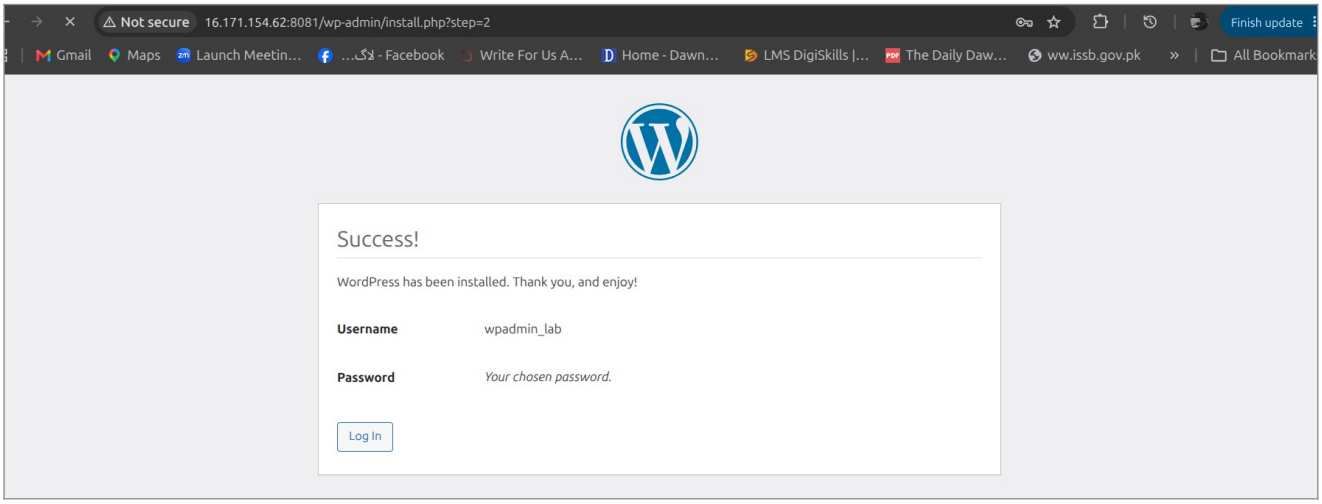


Figure 41: *WordPress installation completed successfully*

The student logs in with the new administrator account and reaches the WordPress dashboard.

Address opened in the browser

```
16.171.154.62:8081/wp-admin/
```

Address explanation

- `/wp-admin/` — the WordPress administration area, opened after logging in.

Observation

The dashboard of "Kubernetes Lab Website" opens with the greeting "Howdy, wpadmin_lab" and the "Welcome to WordPress!" panel. A notice offers a newer WordPress version; the site runs version 6.2.1, which matches the image `wordpress:6.2.1-apache` used in the manifest.

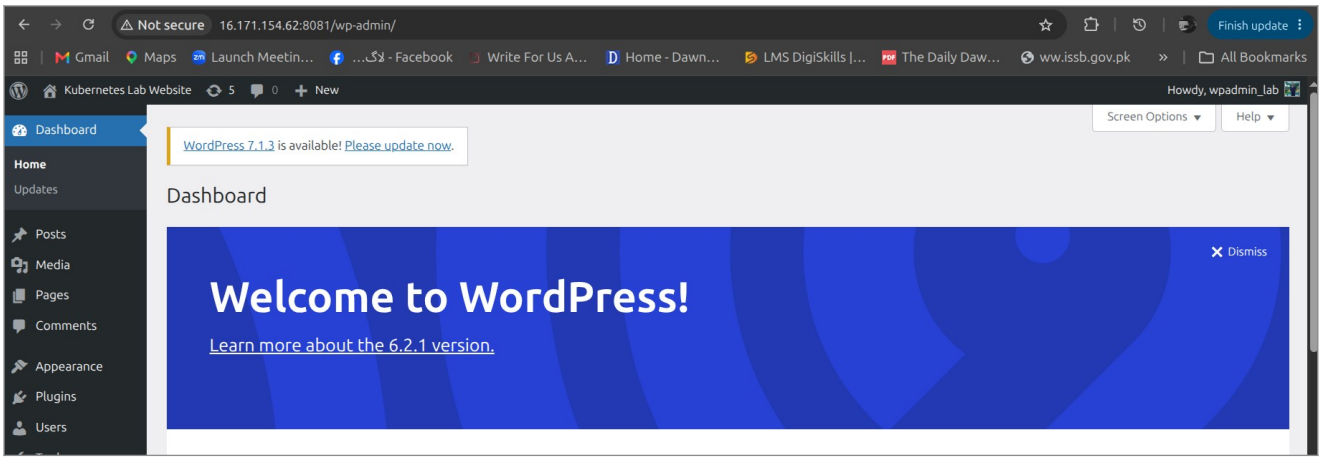


Figure 42: *WordPress dashboard after logging in*

10. Deploying the To-Do List Application

The last application is a to-do list web application, deployed with a Deployment of three replicas and a NodePort service. Using a separate file for each resource follows common Kubernetes practice.

10.1 Creating the To-Do Deployment Manifest

The student creates `deployment.yml` in nano. The file defines the Deployment that runs the to-do application. The screenshot shows the upper part of the file; the lines after the container name are below the visible area of the editor.

Key fields in the file

- `kind: Deployment` and `name: todo-deployment` — the type and name of the resource, with the label `app: todo-app`.
- `replicas: 3` — three identical pods must be kept running.
- `selector.matchLabels` and `template.metadata.labels` — both use `app: todo-app`, so the deployment manages the pods it creates.
- `containers` — the first container is named `todo-app`.

Observation

The editor shows `deployment.yml` with the deployment definition for `todo-deployment`.

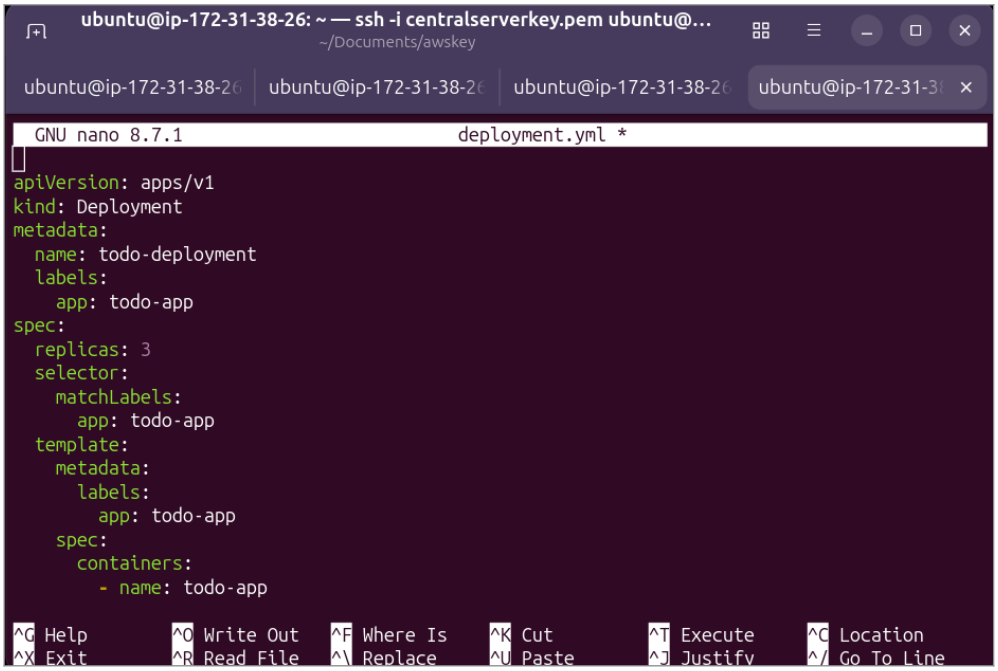


Figure 43: *deployment.yml in nano: todo-deployment with 3 replicas*

10.2 Creating the To-Do Service Manifest

The student creates `service.yml`, which exposes the to-do pods through a NodePort service.

Key fields in the file

- `kind: Service` and `name: todo-service` — the type and name of the resource.
- `type: NodePort` — makes the service available on a port of the node.
- `selector` — `app: todo-app`; selects the pods created by `todo-deployment`.
- `port: 80` — the service port.
- `targetPort: 8000` — the container port that receives the traffic.
- `nodePort: 30007` — the fixed node port assigned to the service.

Observation

The editor shows the complete `service.yml` manifest for `todo-service`.

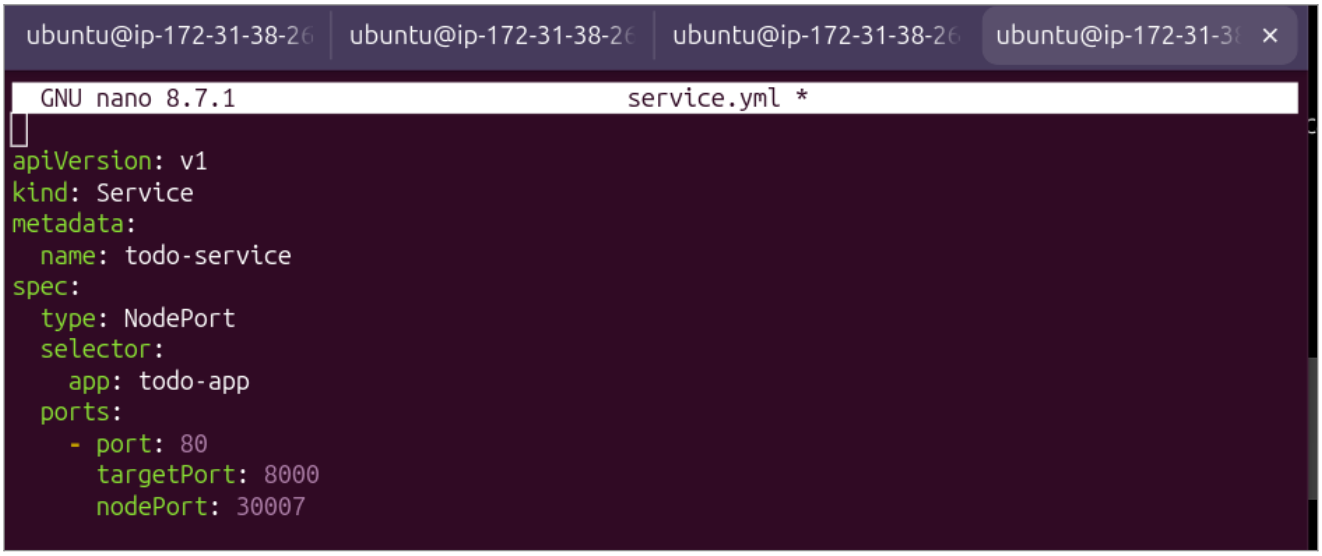


Figure 44: *service.yml in nano: todo-service of type NodePort*

10.3 Applying the To-Do Manifests

The student opens both files in nano and then applies them to the cluster.

Commands used

```
nano deployment.yml
nano service.yml
```

Command explanation

- `nano deployment.yml` and `nano service.yml` → open the two manifest files in the nano editor.

Result

The terminal shows the two editor commands; the files were written in the previous steps.

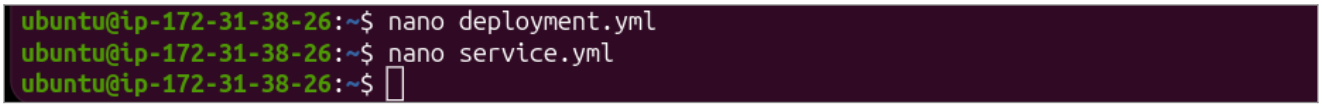


Figure 45: *Opening deployment.yml and service.yml in nano*

The two manifests are applied to the cluster.

Commands used

```
kubectl apply -f deployment.yml
kubectl apply -f service.yml
```

Command explanation

- `kubectl apply -f deployment.yml` → creates the to-do deployment.
- `kubectl apply -f service.yml` → creates the to-do service.

Result

Kubernetes replies `deployment.apps/todo-deployment created` and `service/todo-service created`.

```
ubuntu@ip-172-31-38-26:~$ kubectl apply -f deployment.yml
deployment.apps/todo-deployment created
ubuntu@ip-172-31-38-26:~$ kubectl apply -f service.yml
service/todo-service created
ubuntu@ip-172-31-38-26:~$
```

Figure 46: *Applying deployment.yml and service.yml*

10.4 Verifying the To-Do Application

The student checks the deployment, its pods and the service.

Commands used

```
kubectl get deployment todo-deployment
kubectl get pods -l app=todo-app
kubectl get service todo-service
```

Command explanation

- `kubectl get deployment todo-deployment` → shows the deployment status.
- `kubectl get pods -l app=todo-app` → lists the pods with the label `app=todo-app`.
- `kubectl get service todo-service` → shows the service details.

Result

The deployment shows `3/3` ready. Three `todo-deployment-79ddc7dc7-...` pods are `1/1 Running` with 0 restarts. The service `todo-service` is of type `NodePort` with cluster IP `10.105.145.167` and ports `80:30007/TCP`, which matches the `nodePort` in the manifest.

```
ubuntu@ip-172-31-38-26:~$ kubectl get deployment todo-deployment
NAME              READY   UP-TO-DATE   AVAILABLE   AGE
todo-deployment   3/3     3            3           55s
ubuntu@ip-172-31-38-26:~$ kubectl get pods -l app=todo-app
NAME                               READY   STATUS    RESTARTS   AGE
todo-deployment-79ddc7dc7-9nmcq    1/1     Running   0          65s
todo-deployment-79ddc7dc7-nh5r6    1/1     Running   0          65s
todo-deployment-79ddc7dc7-rlcj4    1/1     Running   0          65s
ubuntu@ip-172-31-38-26:~$ kubectl get service todo-service
NAME           TYPE       CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
todo-service   NodePort   10.105.145.167   <none>        80:30007/TCP   67s
ubuntu@ip-172-31-38-26:~$
```

Figure 47: *To-do deployment, three pods and NodePort service verified*

10.5 Using the To-Do Application in the Browser

The to-do application is opened through the EC2 public IP on port 8083. This port is forwarded to the `todo-service` service; the port-forward terminal for it is shown in Section 11.

Address opened in the browser

```
16.171.154.62:8083/todos/
```

Address explanation

- `16.171.154.62:8083` → public IP and the forwarded port for the to-do service.
- `/todos/` → the path of the to-do page.

Observation

The page titled “Todo List - Modified once again” loads with an input box (showing “Do laundry”) and an Add button, which shows the application is running and reachable.

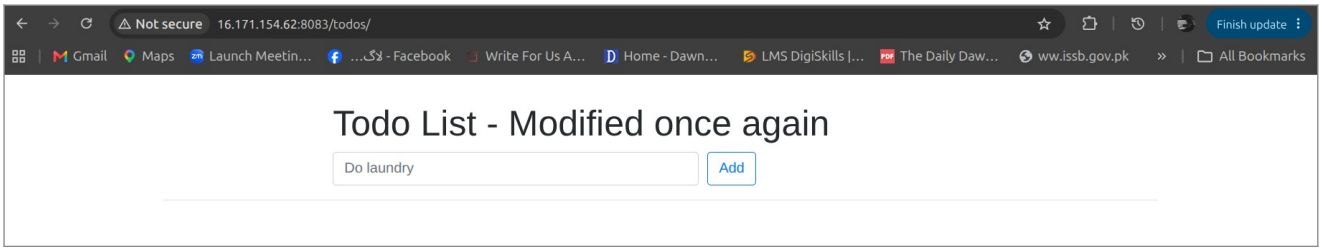


Figure 48: *To-do application opened at 16.171.154.62:8083/todos/*

The student uses the application by adding tasks to the list.

Observation

The list now contains three tasks: “Turn off EC2 Instance”, “Submit German Assignment” and “Upload Lab Task”. The last task is ticked and shown struck through, meaning it is marked as completed.

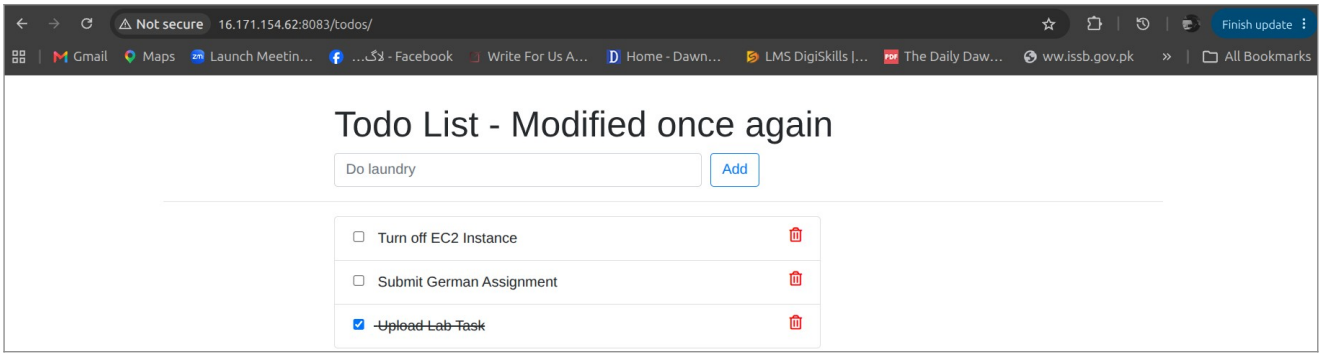


Figure 49: *To-do application showing three tasks, one marked completed*

11. Port-Forward Sessions for All Applications

Each application is made reachable on its own port by a separate `kubectl port-forward` command. Every command keeps running in its own SSH terminal, and the `Handling connection for ...` lines appear whenever the browser connects. The screenshots below show these terminals.

11.1 Nginx on Port 8082

This terminal belongs to the first Nginx deployment created with kubectl commands (Section 7.3).

Command used

```
kubectl port-forward service/nginx 8082:80 --address=0.0.0.0
```

Command explanation

- The command forwards server port 8082 to port 80 of the `nginx` service, listening on all interfaces.

Result

After `Forwarding from 0.0.0.0:8082 -> 80`, two `Handling connection for 8082` lines appear, showing that the browser has connected to the application through this port.

```
ubuntu@ip-172-31-38-26:~$ kubectl port-forward service/nginx 8082:80 --address=0.0.0.0
Forwarding from 0.0.0.0:8082 -> 80
Handling connection for 8082
Handling connection for 8082
```

Figure 50: *Port-forward terminal for the nginx service on port 8082*

11.2 To-Do Application on Port 8083

This terminal serves the to-do application. The service port 80 is forwarded to the container port 8000, as defined by `targetPort` in `service.yml`.

Command used

```
kubectl port-forward service/todo-service 8083:80 --address=0.0.0.0
```

Command explanation

- `service/todo-service` — the service to forward to.
- `8083:80` — server port 8083 is forwarded to port 80 of the service.
- `--address=0.0.0.0` — listens on all network interfaces.

Result

The terminal prints `Forwarding from 0.0.0.0:8083 -> 8000`, so traffic arriving on port 8083 is delivered to port 8000 of the to-do pods.

```
ubuntu@ip-172-31-38-26:~$ kubectl port-forward service/todo-service 8083:80 --address=0.0.0.
0
Forwarding from 0.0.0.0:8083 -> 8000
```

Figure 51: *Starting the port forward for the to-do service on port 8083*

11.3 Nginx (YAML) on Port 8080

This terminal serves the Nginx application deployed from the YAML manifests (Section 8.5).

Command used

```
kubectl port-forward service/nginx-svc 8080:80 --address=0.0.0.0
```

Command explanation

- The command forwards server port 8080 to port 80 of the `nginx-svc` service.

Result

After `Forwarding from 0.0.0.0:8080 -> 80`, three `Handling connection for 8080` lines show that the application was opened from the browser.

```
ubuntu@ip-172-31-38-26:~$ kubectl port-forward service/nginx-svc 8080:80 --address=0.0.0.0
Forwarding from 0.0.0.0:8080 -> 80
Handling connection for 8080
Handling connection for 8080
Handling connection for 8080
```

Figure 52: *Port-forward terminal for the nginx-svc service on port 8080*

11.4 WordPress on Port 8081

This terminal serves the WordPress site (Section 9.5).

Command used

```
kubectl port-forward service/wordpress-frontend 8081:80 --address=0.0.0.0
```

Command explanation

- The command forwards server port 8081 to port 80 of the `wordpress-frontend` service.

Result

After `Forwarding from 0.0.0.0:8081 -> 80`, several `Handling connection for 8081` lines appear. WordPress loads many pages and files, so it creates more connections than the other applications.

```
ubuntu@ip-172-31-38-26:~$ kubectl port-forward service/wordpress-frontend 8081:80 --address=
0.0.0.0
Forwarding from 0.0.0.0:8081 -> 80
Handling connection for 8081
Handling connection for 8081
Handling connection for 8081
```

Figure 53: *Port-forward terminal for the wordpress-frontend service on port 8081*

11.5 To-Do Application with Active Connections

The to-do port-forward command is shown again, this time after the application has been used in the browser.

Command used

```
kubectl port-forward service/todo-service 8083:80 --address=0.0.0.0
```

Command explanation

- The command is the same as in Section 11.2.

Result

After `Forwarding from 0.0.0.0:8083 -> 8000`, several `Handling connection for 8083` lines appear, showing that browser requests are being received by the to-do application.

```
ubuntu@ip-172-31-38-26:~$ kubectl port-forward service/todo-service 8083:80 --address=0.0.0.
0
Forwarding from 0.0.0.0:8083 -> 8000
Handling connection for 8083
Handling connection for 8083
Handling connection for 8083
```

Figure 54: *Port-forward terminal for the to-do service with active connections*

11.6 All Applications Running Together

With all port-forward terminals active, the four applications are opened in separate browser windows, each through the same EC2 public IP and a different port.

Windows shown

- `16.171.154.62:8082` → "Welcome to nginx!" (Nginx created with kubectl commands).
- `16.171.154.62:8083/todos/` → the to-do list application.
- `16.171.154.62:8080` → "Welcome to nginx!" (Nginx created from YAML manifests).
- `16.171.154.62:8081/wp-a...` → the WordPress dashboard.

Observation

All four applications are running in the same Kubernetes cluster and can be opened at the same time, each on its own port.

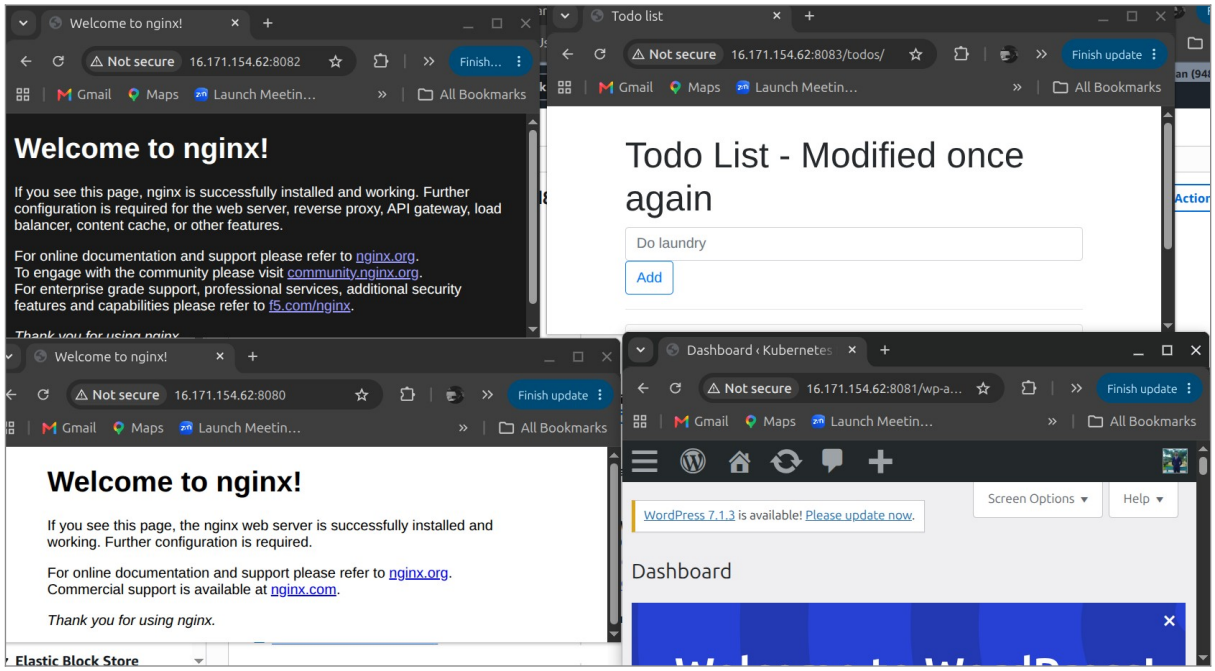


Figure 55: *Nginx (8082 and 8080), to-do list (8083) and WordPress dashboard (8081) open together*

12. Final Verification of the Cluster

At the end of the lab, the student reviews the whole cluster to confirm that every workload is still running.

12.1 Nodes and Deployments

The student checks the node and then lists all deployments.

Commands used

```
kubectl get nodes
kubectl get deployments
```

Command explanation

- `kubectl get nodes` → shows the status of the cluster node.

- `kubectl get deployments` — lists all deployments with their ready replicas.

Result

The node `minikube` is `Ready` with the role `control-plane` and version `v1.37.0`. The deployments shown are `nginx` (1/1), `nginx-dep` (4/4), `todo-deployment` (3/3), `wordpress-frontend` (1/1) and `wordpress-mysql` (1/1). The screenshot is cut at the bottom of the terminal after the `wordpress-mysql` line.

```
ubuntu@ip-172-31-38-26:~$ kubectl get nodes
NAME       STATUS   ROLES           AGE   VERSION
minikube   Ready    control-plane   73m   v1.37.0
ubuntu@ip-172-31-38-26:~$ kubectl get deployments
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
nginx               1/1     1            1           70m
nginx-dep           4/4     4            4           57m
todo-deployment     3/3     3            3           21m
wordpress-frontend  1/1     1            1           44m
wordpress-mysql     1/1     1            1           44m
```

Figure 56: *Final check of the node and all deployments*

12.2 Services and Pods

The student lists all services and all pods for a final overview of the cluster.

Commands used

```
kubectl get services
kubectl get pods
```

Command explanation

- `kubectl get services` — lists all services with their type, IP address and ports.
- `kubectl get pods` — lists all pods with their status.

Result

Six services are listed: `kubernetes` (ClusterIP), `nginx` (NodePort, `80:30131/TCP`), `nginx-svc` (LoadBalancer, `80:30405/TCP`), `todo-service` (NodePort, `80:30007/TCP`), `wordpress-frontend` (LoadBalancer, `80:31766/TCP`) and `wordpress-mysql` (ClusterIP, `3306/TCP`). Ten pods are listed: one nginx pod, four `nginx-dep` pods, three `todo-deployment` pods, one `wordpress-frontend` pod and one `wordpress-mysql` pod. All are `1/1 Running` with 0 restarts.

```
ubuntu@ip-172-31-38-26:~$ kubectl get services
NAME                TYPE           CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
kubernetes          ClusterIP      10.96.0.1        <none>        443/TCP        74m
nginx               NodePort       10.111.198.135   <none>        80:30131/TCP   70m
nginx-svc           LoadBalancer   10.109.177.192   <pending>     80:30405/TCP   57m
todo-service        NodePort       10.105.145.167   <none>        80:30007/TCP   21m
wordpress-frontend  LoadBalancer   10.111.210.19    <pending>     80:31766/TCP   44m
wordpress-mysql     ClusterIP      10.111.191.104   <none>        3306/TCP       45m
ubuntu@ip-172-31-38-26:~$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
nginx-598cc96cd9-jhf89              1/1     Running   0          70m
nginx-dep-5fd6b8bd65-bmb9g          1/1     Running   0          58m
nginx-dep-5fd6b8bd65-m98rl          1/1     Running   0          58m
nginx-dep-5fd6b8bd65-w6jw8          1/1     Running   0          50m
nginx-dep-5fd6b8bd65-z5q95          1/1     Running   0          58m
todo-deployment-79ddc7dc7-9nmcq     1/1     Running   0          21m
todo-deployment-79ddc7dc7-nh5r6     1/1     Running   0          21m
todo-deployment-79ddc7dc7-rlcj4     1/1     Running   0          21m
wordpress-frontend-6c76bd44-bwcx8   1/1     Running   0          45m
wordpress-mysql-5bc89547d7-tm76v    1/1     Running   0          45m
ubuntu@ip-172-31-38-26:~$
```

Figure 57: *Final check of all services and pods*

12.3 Self-Healing Test: Deleting a Pod Again

To demonstrate self-healing once more, the student first lists the four pods of the `nginx-dep` deployment.

Command used

```
kubectl get pods -l app=nginx-kube
```

Command explanation

- `kubectl get pods -l app=nginx-kube` → lists the pods with the label `app=nginx-kube`.

Result

Four pods are running: `bmb9g`, `m98rl`, `w6jw8` and `z5q95` (name endings), each `1/1 Running` with 0 restarts.

```
ubuntu@ip-172-31-38-26:~$ kubectl get pods -l app=nginx-kube
NAME                        READY   STATUS    RESTARTS   AGE
nginx-dep-5fd6b8bd65-bmb9g  1/1     Running   0          61m
nginx-dep-5fd6b8bd65-m98rl  1/1     Running   0          61m
nginx-dep-5fd6b8bd65-w6jw8  1/1     Running   0          53m
nginx-dep-5fd6b8bd65-z5q95  1/1     Running   0          61m
ubuntu@ip-172-31-38-26:~$
```

Figure 58: *The four nginx-dep pods before the deletion test*

The student deletes one of these pods, `nginx-dep-5fd6b8bd65-bmb9g`.

Command used

```
kubectl delete pod nginx-dep-5fd6b8bd65-bmb9g
```

Command explanation

- `kubectl delete pod nginx-dep-5fd6b8bd65-bmb9g` → deletes the pod with this name.

Result

Kubernetes confirms `pod "nginx-dep-5fd6b8bd65-bmb9g" deleted from default namespace`.

```
ubuntu@ip-172-31-38-26:~$ kubectl delete pod nginx-dep-5fd6b8bd65-bmb9g
pod "nginx-dep-5fd6b8bd65-bmb9g" deleted from default namespace
ubuntu@ip-172-31-38-26:~$
```

Figure 59: *Deleting the pod nginx-dep-5fd6b8bd65-bmb9g*

The student watches the pod list to see how Kubernetes reacts to the deletion.

Command used

```
kubectl get pods -l app=nginx-kube -w
```

Command explanation

- `kubectl get pods -l app=nginx-kube` → lists the pods of the deployment.
- `-w` → "watch" mode; the command keeps running and prints any change to the pods.

Result

The deleted pod `bmb9g` is no longer listed. A new pod, `nginx-dep-5fd6b8bd65-qx9tf`, is running with an age of 27 seconds, and the other three pods are unchanged. There are again four running pods, which confirms that Kubernetes automatically replaced the deleted pod to keep the desired number of replicas.

```
ubuntu@ip-172-31-38-26:~$ kubectl get pods -l app=nginx-kube -w
NAME                         READY   STATUS    RESTARTS   AGE
nginx-dep-5fd6b8bd65-m98rl   1/1     Running   0          63m
nginx-dep-5fd6b8bd65-qx9tf   1/1     Running   0          27s
nginx-dep-5fd6b8bd65-w6jw8   1/1     Running   0          55m
nginx-dep-5fd6b8bd65-z5q95   1/1     Running   0          63m
```

Figure 60: *Watching Kubernetes create the replacement pod qx9tf*

13. Conclusion

In this lab, a Kubernetes cluster was created with Minikube on an Ubuntu AWS EC2 instance, after installing Docker, Minikube and kubectl. Nginx was deployed with kubectl commands and with YAML manifests, a WordPress and MySQL application and a to-do list application were deployed, and all applications were made reachable from a browser using `kubectl port-forward`. The tests in Sections 8.6 and 12.3 showed that Kubernetes automatically replaces a deleted pod to keep the desired number of replicas. The table below summarises the applications deployed in this lab.

Application	Kubernetes objects	Service	Browser port
Nginx (kubectl commands)	Deployment nginx (1 replica)	nginx, NodePort 80:30131	8082
Nginx (YAML manifests)	Deployment nginx-dep (4 replicas)	nginx-svc, LoadBalancer 80:30405	8080
WordPress + MySQL	Deployments wordpress-frontend and wordpress-mysql (1 replica each)	wordpress-frontend, LoadBalancer 80:31766; wordpress-mysql, ClusterIP 3306	8081
To-do list	Deployment todo-deployment (3 replicas)	todo-service, NodePort 80:30007	8083